

Shortest Path Algorithms

A comprehensive guide to shortest path algorithms for SDE placements, online assessments, and competitive programming.

BFS Shortest Path in Unweighted Graph

1. Overview

BFS (Breadth-First Search) finds the shortest path in an **unweighted graph** — a graph where every edge has the same cost (typically 1). It explores the graph level by level, and the first time a node is reached, that path is guaranteed to be the shortest in terms of number of edges.

2. Intuition

Imagine you are in a maze and want to find the shortest route to an exit. You start at the entrance and take one step in every possible direction. If you haven't found the exit, you take a second step from each position you reached. The moment you step on the exit, you know you've taken the fewest possible steps — because you explored all paths of length 1 first, then length 2, then length 3, and so on.

Why BFS works for shortest paths: BFS uses a queue. When you pop a node, you push all its unvisited neighbors. Since the queue is FIFO, nodes at distance d are always processed before nodes at distance $d+1$. This guarantees that the first time you discover a node, you have reached it via the shortest path.

3. When to Use It

- Graph is **unweighted** (all edges have equal weight, usually 1)
- Need shortest path in terms of **number of edges** (or hops)
- Single-source shortest path (SSSP) in an unweighted graph
- Grid-based shortest path problems (maze, 2D matrix)
- Word ladder problems (transforming one word to another)
- Finding minimum moves in a game (chess knight, etc.)
- Level-order traversal applications

Trigger phrases: "shortest path in unweighted graph", "minimum number of moves", "minimum steps", "shortest distance in a grid", "word ladder", "level order", "fewest edges"

4. When Not to Use It

- **Weighted graph** — BFS does not consider edge weights and will fail to find the shortest path
- **Large state space with high branching factor** — BFS can exhaust memory
- **Need to find all shortest paths** — BFS finds one shortest path; use modified BFS or DP for all paths
- **Graph is huge but the target is deep** — BFS explores level by level, so if the target is far, it visits many nodes; DFS might be better for reachability (not shortest)
- *Single pair shortest path when A is applicable** — A* is more efficient with a good heuristic

5. Core Concepts

5.1 Queue

The core data structure. BFS uses a FIFO queue to process nodes in order of discovery. This ensures level-by-level traversal.

5.2 Visited Array / Distance Array

- **Visited array:** Prevents revisiting nodes (important in cyclic graphs)
- **Distance array:** Stores the shortest distance from source to each node. Initialized to `-1` or `INF`, and updated when a node is first discovered

5.3 Level-by-Level Traversal

BFS processes all nodes at distance `d` before any node at distance `d+1`. This is the key property that guarantees shortest paths in unweighted graphs.

5.4 Adjacency List / Matrix

The graph representation. Adjacency list is preferred for sparse graphs (most CP problems). Adjacency matrix is simpler but uses $O(V^2)$ memory.

5.5 Parent Array (for path reconstruction)

An array `parent[v]` stores the node from which `v` was first reached. To reconstruct the path, trace back from destination to source using the parent array.

6. Step-by-Step Algorithm

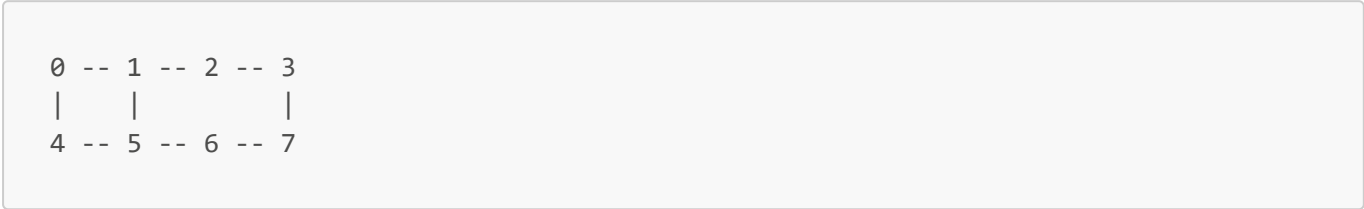
Input: Graph $G(V, E)$, source s , destination t
Output: Shortest distance from s to t , and the path

1. Initialize:
 - `dist[v] = INF` for all v in V , `dist[s] = 0`
 - `parent[v] = -1` for all v in V
 - queue q
 - `q.push(s)`
2. While queue is not empty:
 - a. `u = q.front(), q.pop()`
 - b. For each neighbor v of u :
 - If `dist[v] == INF` (not visited):
 - `dist[v] = dist[u] + 1`
 - `parent[v] = u`
 - `q.push(v)`
 - If `v == t` (optional early exit): break
3. `dist[t]` is the shortest distance.
If `dist[t] == INF`, no path exists.

4. To reconstruct path:
 - Start from t, follow parent[] back to s
 - Reverse the path

7. Dry Run

Graph:



Edges: 0-1, 0-4, 1-2, 1-5, 2-3, 3-7, 4-5, 5-6, 6-7

Source = 0, Destination = 7

Step	Queue (front → back)	Node Popped	Neighbors	Distance Array	Parent Array
Init	[0]	—	—	dist[0]=0, rest=INF	parent[0]=-1
1	[0]	0	1, 4	dist[1]=1, dist[4]=1	parent[1]=0, parent[4]=0
2	[1, 4]	1	2, 5	dist[2]=2, dist[5]=2	parent[2]=1, parent[5]=1
3	[4, 2, 5]	4	5 (already visited)	—	—
4	[2, 5]	2	3	dist[3]=3	parent[3]=2
5	[5, 3]	5	6	dist[6]=3	parent[6]=5
6	[3, 6]	3	7	dist[7]=4	parent[7]=3
7	[6, 7]	6	7 (already visited)	—	—
8	[7]	7	—	Done	—

Shortest distance: dist[7] = 4

Path: 7 ← 3 ← 2 ← 1 ← 0 → Path = [0, 1, 2, 3, 7]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
```

```
// BFS Shortest Path in Unweighted Graph
// Returns distance from source to all nodes
// Reconstructs path to a specific destination

vector<int> bfs_shortest_path(const vector<vector<int>>& adj, int src, int dest,
vector<int>& path) {
    int n = adj.size();
    vector<int> dist(n, INT_MAX);
    vector<int> parent(n, -1);
    queue<int> q;

    dist[src] = 0;
    q.push(src);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        // Early exit if we reached destination
        if (u == dest) break;

        for (int v : adj[u]) {
            if (dist[v] == INT_MAX) {
                dist[v] = dist[u] + 1;
                parent[v] = u;
                q.push(v);
            }
        }
    }

    // Reconstruct path
    if (dist[dest] == INT_MAX) return dist; // No path

    int cur = dest;
    while (cur != -1) {
        path.push_back(cur);
        cur = parent[cur];
    }
    reverse(path.begin(), path.end());

    return dist;
}

// Example usage
int main() {
    int n = 8;
    vector<vector<int>> adj(n);

    // Build graph
    adj[0] = {1, 4};
    adj[1] = {0, 2, 5};
    adj[2] = {1, 3};
    adj[3] = {2, 7};
}
```

```

adj[4] = {0, 5};
adj[5] = {1, 4, 6};
adj[6] = {5, 7};
adj[7] = {3, 6};

int src = 0, dest = 7;
vector<int> path;
vector<int> dist = bfs_shortest_path(adj, src, dest, path);

cout << "Shortest distance: " << dist[dest] << "\n";
cout << "Path: ";
for (int v : path) cout << v << " ";
cout << "\n";

return 0;
}

```

9. Python Implementation

```

from collections import deque

def bfs_shortest_path(adj, src, dest):
    n = len(adj)
    dist = [float('inf')] * n
    parent = [-1] * n
    q = deque()

    dist[src] = 0
    q.append(src)

    while q:
        u = q.popleft()

        if u == dest:
            break

        for v in adj[u]:
            if dist[v] == float('inf'):
                dist[v] = dist[u] + 1
                parent[v] = u
                q.append(v)

    # Reconstruct path
    path = []
    if dist[dest] != float('inf'):
        cur = dest
        while cur != -1:
            path.append(cur)
            cur = parent[cur]
        path.reverse()

```

```
        return dist[dest], path

# Example usage
if __name__ == "__main__":
    n = 8
    adj = [[] for _ in range(n)]
    adj[0] = [1, 4]
    adj[1] = [0, 2, 5]
    adj[2] = [1, 3]
    adj[3] = [2, 7]
    adj[4] = [0, 5]
    adj[5] = [1, 4, 6]
    adj[6] = [5, 7]
    adj[7] = [3, 6]

    src, dest = 0, 7
    dist, path = bfs_shortest_path(adj, src, dest)

    print(f"Shortest distance: {dist}")
    print(f"Path: {' '.join(map(str, path))}")
```

10. Code Explanation

- **dist array:** Initialized to `INT_MAX` (or `inf`). `dist[u]` stores the shortest distance from source to `u`. Set `dist[src] = 0`.
- **parent array:** Tracks the predecessor of each node for path reconstruction. Initialize to `-1`.
- **Queue:** A standard queue stores nodes to process. FIFO order ensures level-by-level traversal.
- **Main loop:** Pop node `u`, examine all its neighbors `v`. If `dist[v]` is unvisited, set `dist[v] = dist[u] + 1`, record `parent[v] = u`, and push `v`.
- **Early exit:** If the destination is found, break out of the loop (optimization).
- **Path reconstruction:** Start from `dest`, follow `parent` pointers back to `src`, then reverse the list.
- **No path case:** If `dist[dest]` remains `INT_MAX`, the destination is unreachable.

11. Complexity Analysis

Operation	Complexity
Time Complexity	O(V + E) — each vertex is enqueued once, each edge is examined once
Space Complexity	O(V) — for queue, visited, dist, parent arrays
Path Reconstruction	O(V) in worst case
Grid BFS	O(R × C) — exactly one cell per row and column

- **Best case:** $O(V + E)$ — you always traverse the whole graph or early exit at the target
- **Worst case:** $O(V + E)$ — the graph is fully traversed

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Grid BFS	2D grid, moves in 4/8 directions	Use delta arrays: <code>dx = {0,0,1,-1}</code> , <code>dy = {1,-1,0,0}</code> ; treat each cell as a node	Rotting Oranges, Shortest Path in Grid
Multi-source BFS	Multiple starting points (e.g., multiple gates, rotten oranges)	Push all sources into queue with <code>dist=0</code> initially	Walls and Gates, 01 Matrix
Word Ladder	Transform one word to another, one letter change per step	Each word is a node, edges between words differing by 1 letter	Word Ladder (LeetCode 127)
BFS on Matrix with Obstacles	Grid with walls, find shortest path	Skip blocked cells, BFS on valid cells	Shortest Path in Binary Matrix
Level-order BFS	Need to know depth or process level by level	Process queue in batches: <code>for (int sz = q.size(); sz > 0; sz--)</code>	Binary Tree Level Order, Min Depth of Tree
BFS with State	Need to track more than just position (e.g., keys, visited mask)	Use a visited array with state dimension: <code>visited[node][state]</code>	Shortest Path with Keys, Sliding Puzzle

13. Common Mistakes

- **Not marking visited on push** — If you mark visited only on pop, nodes can be pushed multiple times, causing exponential blowup
- **Using `visited` boolean instead of `dist` array** — Using a separate boolean array + distance array is redundant; use `dist` initialized to `-1/INF` as both visited tracker and distance store
- **Forgetting path reversal** — Path reconstruction from parent pointers gives reversed path; don't forget to `reverse()`
- **Infinite loop in cyclic graph** — Always mark visited before pushing, or use a visited check
- **Not handling disconnected graphs** — If destination is unreachable, `dist[dest]` remains `INF`; handle this case
- **Using recursion** — BFS is iterative; don't use recursion (that's DFS)
- **Off-by-one in grid BFS** — Check bounds properly: `0 <= nx < rows && 0 <= ny < cols`
- **Using `int` for distance when path can be long** — Use `INT_MAX / 1e9` as `INF`, not a magic number

14. Edge Cases

- **Source = destination** → distance = 0, path = [src]
- **Disconnected graph** → destination unreachable, return `INF / -1`
- **Empty graph (0 nodes)** → handle gracefully
- **Single node graph** → BFS works fine
- **Graph with self-loops** → BFS should not loop infinitely if visited is properly tracked
- **Complete graph** → BFS visits all nodes in one level; $O(V^2)$ edges
- **Grid with blocked start/end** → if start or end is blocked, return `-1`
- **Large graph with millions of nodes** → BFS may run out of memory; consider bidirectional BFS

15. Variations

Variation	What Changes	When Used	Importance
Bidirectional BFS	Run BFS from both source and destination simultaneously; stop when frontiers meet	When both source and target are known and graph is large	High — reduces search space from $O(V)$ to $O(V^{1/2})$
0-1 BFS	Uses deque instead of queue; edges with weight 0 or 1	Weighted graphs with only 0/1 weights	Covered separately below
Multi-source BFS	Push all sources into queue initially	Problems with multiple starting points	High — common pattern
BFS with State	3D visited array: <code>visited[node][state]</code>	Problems with state/mask/keys	Medium — seen in harder problems
BFS on Implicit Graph	Graph is not given explicitly; generated on the fly	Puzzles, sliding puzzles, Rubik's cube	Important for hard problems

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
DFS	Also traverses graph; does not guarantee shortest path	Use DFS for connectivity, topological sort, cycle detection; use BFS for shortest paths in unweighted graphs
Dijkstra	Generalization of BFS for weighted graphs	Use when edges have non-negative weights
0-1 BFS	Special case of Dijkstra for weights 0 or 1	Use when weights are only 0 or 1 (faster than Dijkstra)
A*	Informed search using heuristic; BFS with a priority queue	Use when you have a good heuristic and need shortest path in a large graph

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Shortest Path in Binary Matrix	LeetCode 1091	Grid BFS with 8-directional moves	Easy
Minimum Depth of Binary Tree	LeetCode 111	BFS level-order traversal	Easy

Medium

Problem	Platform	Pattern	Difficulty
Word Ladder	LeetCode 127	BFS with implicit graph (word transformation)	Medium

Problem	Platform	Pattern	Difficulty
Rotting Oranges	LeetCode 994	Multi-source BFS	Medium
01 Matrix	LeetCode 542	Multi-source BFS from all 0s	Medium

Hard

Problem	Platform	Pattern	Difficulty
Shortest Path Visiting All Nodes	LeetCode 847	BFS with bitmask state	Hard
Sliding Puzzle	LeetCode 773	BFS on state space (implicit graph)	Hard
Word Ladder II	LeetCode 126	BFS + DFS for all shortest paths	Hard

18. Interview Explanation

"BFS finds the shortest path in an unweighted graph by exploring nodes level by level. Starting from the source, we push all unvisited neighbors into a queue. Since queues are FIFO, nodes at distance d are always processed before nodes at distance $d+1$. The first time we reach a node, we have the shortest path to it. We use a distance array initialized to infinity, set $\text{dist}[\text{source}] = 0$, and when we pop a node u and find a neighbor v that hasn't been visited, we set $\text{dist}[v] = \text{dist}[u] + 1$ and push v . This gives $O(V + E)$ time and $O(V)$ space. For path reconstruction, we maintain a parent array to trace back from destination to source."

19. Revision Notes

- **Key idea:** Level-by-level traversal using a FIFO queue
- **Guarantee:** First time a node is discovered = shortest path (unweighted)
- **Time:** $O(V + E)$
- **Space:** $O(V)$
- **Data structures:** Queue, distance array, parent array (optional)
- **Grid BFS:** Use dx/dy delta arrays, check bounds
- **Multi-source:** Push all sources into queue initially
- **Bidirectional BFS:** Run from both ends, meet in middle
- **Common trap:** Mark visited *when pushing to queue*, not when popping
- **Path reconstruction:** Reverse after following parent pointers

20. Final Cheat Sheet

BFS SHORTEST PATH (UNWEIGHTED)

WHEN: Unweighted graph, shortest path by edge count

COMPLEXITY: $O(V + E)$ time, $O(V)$ space

DATA STRUCTURE: Queue

TEMPLATE:

```
dist[src] = 0
q.push(src)
```

```
while (!q.empty()):
    u = q.front(); q.pop()
    for v in adj[u]:
        if dist[v] == INF:
            dist[v] = dist[u] + 1
            parent[v] = u
            q.push(v)
```

KEY RULE: Mark visited on push, not on pop

EDGE CASES: src == dest, disconnected graph, blocked cells

PATH: Follow parent[] from dest to src, then reverse

Dijkstra's Algorithm

1. Overview

Dijkstra's algorithm finds the **shortest path from a single source** to all other nodes in a **weighted graph with non-negative edge weights**. It is one of the most fundamental algorithms in graph theory and a must-know for any placement interview.

2. Intuition

Imagine you are at the center of a city and want to find the shortest driving distance to every other location. You start knowing the distance to your current location is 0. You look at all the roads from your current location and note the distances to each neighboring intersection. You then go to the **closest unvisited intersection**. From there, you update your knowledge: if going through this intersection gives a shorter path to any other place, you update your map. You repeat this — always going to the nearest unvisited intersection — until you have visited all intersections.

Why it works: Dijkstra's algorithm is a **greedy algorithm**. At each step, it picks the unvisited node with the smallest known distance. Because all edges have non-negative weights, once a node is visited (its distance is finalized), no shorter path to it can be found later — any alternate path would have to go through an unvisited node with a larger distance, making it longer.

3. When to Use It

- **Single-source shortest path** in a graph with **non-negative edge weights**
- **Weighted graph** where edge weights are positive or zero
- **Road networks, maps, GPS navigation** (real-world distances are non-negative)
- **Network routing protocols** (OSPF, IS-IS)
- **Problems where you need the shortest path from one source to all nodes**
- **Problems where the graph is sparse** (use adjacency list + priority queue)

Trigger phrases: "shortest path in weighted graph", "minimum cost to reach", "cheapest route", "lowest total weight", "non-negative weights", "find shortest distance from source"

4. When Not to Use It

- **Graph with negative edge weights** — Dijkstra will fail (produce incorrect results). Use Bellman-Ford instead.
- **All-pairs shortest path** — Running Dijkstra from each node gives $O(V(E + V \log V))$; use Floyd-Warshall for dense graphs ($O(V^3)$)
- **Unweighted graph** — BFS is simpler and faster ($O(V + E)$ vs $O((V + E) \log V)$)
- **Graph with only 0/1 weights** — Use 0-1 BFS ($O(V + E)$) instead
- **Dense graph** — For dense graphs, simple Dijkstra (without heap, $O(V^2)$) may be faster than heap-based $O((V + E) \log V)$
- **Need shortest paths from all nodes to all nodes** — Use Floyd-Warshall or Johnson's algorithm

5. Core Concepts

5.1 Relaxation

The core operation: if $\text{dist}[u] + \text{weight}(u, v) < \text{dist}[v]$, update $\text{dist}[v] = \text{dist}[u] + \text{weight}(u, v)$. This is called "relaxing" the edge.

5.2 Priority Queue (Min-Heap)

Used to efficiently get the unvisited node with the smallest distance. In C++, `priority_queue` with `greater<pair<int,int>>` gives a min-heap.

5.3 Visited / Finalized Set

Once a node is popped from the priority queue, its distance is finalized. This is because all remaining distances in the heap are $\geq$ the popped node's distance (due to non-negative weights).

5.4 Lazy Deletion

A node may be pushed into the priority queue multiple times with different distances (when a shorter path is found). When we pop a stale entry (distance $>$ current $\text{dist}[\text{node}]$), we skip it. This is called lazy deletion.

5.5 Shortest Path Tree (SPT)

The set of edges used to reach each node via the shortest path forms a tree rooted at the source.

6. Step-by-Step Algorithm

Input: Graph $G(V, E)$ with non-negative weights, source s
 Output: Shortest distance from s to all nodes

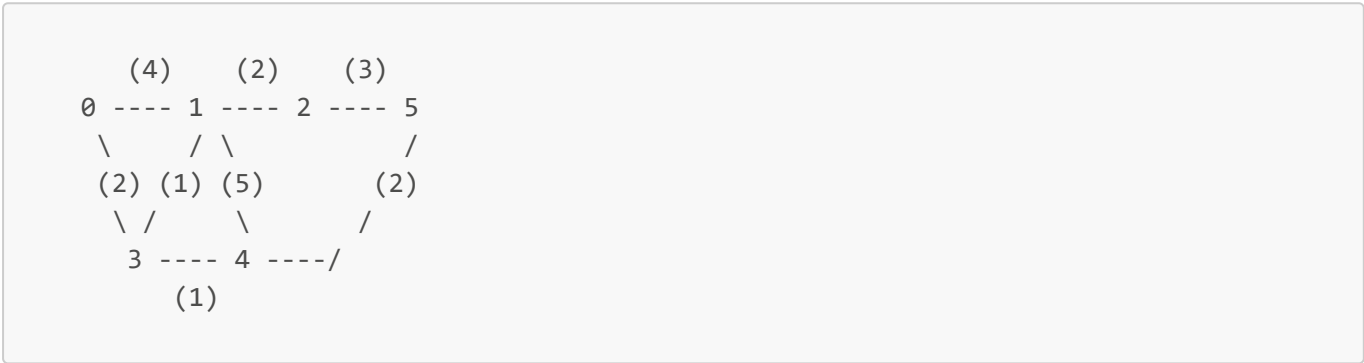
1. Initialize:
 - $\text{dist}[v] = \text{INF}$ for all v in V , $\text{dist}[s] = 0$
 - $\text{parent}[v] = -1$ for all v
 - min-heap pq of (distance, node) pairs
 - $\text{pq.push}(\{0, s\})$
2. While pq is not empty:
 - a. Pop (d, u) from pq
 - b. If $d > \text{dist}[u]$: continue (lazy deletion — stale entry)

```
c. For each neighbor v of u with weight w:
  - If dist[u] + w < dist[v]:
    - dist[v] = dist[u] + w
    - parent[v] = u
    - pq.push({dist[v], v})

3. dist[] now contains shortest distances from source.
   For unreachable nodes, dist[v] = INF.
```

7. Dry Run

Graph:



Edges with weights: 0→1(4), 0→3(2) 1→2(2), 1→3(1), 1→4(5) 2→5(3) 3→4(1) 4→5(2)

Source = 0

Step	PQ (dist, node)	Pop	dist array	Processed neighbors
Init	(0, 0)	—	[0, ∞, ∞, ∞, ∞, ∞]	—
1	(0, 0)	0	[0, 4, ∞, 2, ∞, ∞]	1(dist=4), 3(dist=2)
2	(2, 3), (4, 1)	3	[0, 4, ∞, 2, 3, ∞]	4(dist=2+1=3)
3	(3, 4), (4, 1)	4	[0, 4, ∞, 2, 3, 5]	5(dist=3+2=5)
4	(4, 1), (5, 5)	1	[0, 4, 6, 2, 3, 5]	2(dist=4+2=6), 3(already visited)
5	(5, 5), (6, 2)	5	[0, 4, 6, 2, 3, 5]	—
6	(6, 2)	2	[0, 4, 6, 2, 3, 5]	5(6+3=9 > 5, skip)

Final distances: [0, 4, 6, 2, 3, 5]

Paths:

- 0→1: [0, 1] (cost 4)
- 0→2: [0, 1, 2] (cost 6)
- 0→3: [0, 3] (cost 2)
- 0→4: [0, 3, 4] (cost 3)
- 0→5: [0, 3, 4, 5] (cost 5)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using pii = pair<int, int>;

const ll INF = 1e18;

// Dijkstra's Algorithm
// Returns vector of shortest distances from source to all nodes
// Also returns parent array for path reconstruction

vector<ll> dijkstra(const vector<vector<pii>>& adj, int src, vector<int>& parent)
{
    int n = adj.size();
    vector<ll> dist(n, INF);
    parent.assign(n, -1);

    // Min-heap: (distance, node)
    priority_queue<pii, vector<pii>, greater<pii>> pq;

    dist[src] = 0;
    pq.push({0, src});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();

        // Lazy deletion: skip stale entries
        if (d != dist[u]) continue;

        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                parent[v] = u;
                pq.push({dist[v], v});
            }
        }
    }

    return dist;
}

// Reconstruct path from source to destination
vector<int> get_path(const vector<int>& parent, int dest) {
    vector<int> path;
    int cur = dest;
    while (cur != -1) {
        path.push_back(cur);
        cur = parent[cur];
    }
}
```

```

        reverse(path.begin(), path.end());
        return path;
    }

    // Example usage
    int main() {
        int n = 6;
        vector<vector<pii>> adj(n);

        // Build graph: adj[u] = vector of {v, weight}
        adj[0] = {{1, 4}, {3, 2}};
        adj[1] = {{2, 2}, {3, 1}, {4, 5}};
        adj[2] = {{5, 3}};
        adj[3] = {{4, 1}};
        adj[4] = {{5, 2}};
        adj[5] = {};

        int src = 0;
        vector<int> parent;
        vector<ll> dist = dijkstra(adj, src, parent);

        cout << "Shortest distances from node " << src << ":\n";
        for (int i = 0; i < n; i++) {
            cout << "    To " << i << ": ";
            if (dist[i] == INF) cout << "INF\n";
            else cout << dist[i] << "\n";
        }

        // Path to node 5
        int dest = 5;
        vector<int> path = get_path(parent, dest);
        cout << "\nPath to " << dest << ": ";
        for (int v : path) cout << v << " ";
        cout << "\n";

        return 0;
    }

```

9. Python Implementation

```

import heapq
from typing import List, Tuple

INF = 10**18

def dijkstra(adj: List[List[Tuple[int, int]]], src: int):
    n = len(adj)
    dist = [INF] * n
    parent = [-1] * n
    dist[src] = 0

```

```

# Min-heap: (distance, node)
pq = [(0, src)]

while pq:
    d, u = heapq.heappop(pq)

    # Lazy deletion: skip stale entries
    if d != dist[u]:
        continue

    for v, w in adj[u]:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            parent[v] = u
            heapq.heappush(pq, (dist[v], v))

return dist, parent

def get_path(parent: List[int], dest: int) -> List[int]:
    path = []
    cur = dest
    while cur != -1:
        path.append(cur)
        cur = parent[cur]
    path.reverse()
    return path

# Example usage
if __name__ == "__main__":
    n = 6
    adj = [[] for _ in range(n)]
    adj[0] = [(1, 4), (3, 2)]
    adj[1] = [(2, 2), (3, 1), (4, 5)]
    adj[2] = [(5, 3)]
    adj[3] = [(4, 1)]
    adj[4] = [(5, 2)]
    adj[5] = []

    src = 0
    dist, parent = dijkstra(adj, src)

    print(f"Shortest distances from node {src}:")
    for i in range(n):
        if dist[i] == INF:
            print(f"  To {i}: INF")
        else:
            print(f"  To {i}: {dist[i]}")

    dest = 5
    path = get_path(parent, dest)
    print(f"\nPath to {dest}: {' '.join(map(str, path))}")

```

10. Code Explanation

- **dist array:** `dist[v]` stores the current known shortest distance from source to `v`. Initialized to `INF`, `dist[src] = 0`.
- **parent array:** Tracks predecessor for path reconstruction. `parent[v] = u` means the shortest path to `v` goes through `u`.
- **Priority queue:** Min-heap of `(distance, node)` pairs. We always process the node with the smallest known distance.
- **Lazy deletion:** When we pop `(d, u)`, if `d != dist[u]`, it means we've already found a shorter path to `u` and this is a stale entry. Skip it.
- **Relaxation:** For each neighbor `v` of `u` with edge weight `w`, if `dist[u] + w < dist[v]`, we've found a shorter path to `v`. Update `dist[v]`, set `parent[v] = u`, and push the new pair into the heap.
- **Path reconstruction:** Follow `parent` pointers from destination back to source, then reverse.

11. Complexity Analysis

Variant	Time Complexity	Space Complexity
Dijkstra with Binary Heap (Adjacency List)	$O((V + E) \log V)$	$O(V)$
Dijkstra with Fibonacci Heap (Adjacency List)	$O(V \log V + E)$	$O(V)$
Dijkstra without Heap (Dense Graph, Adjacency Matrix)	$O(V^2)$	$O(V^2)$
Dijkstra with Set (C++ <code>std::set</code>)	$O((V + E) \log V)$	$O(V)$

- **Best case:** $O((V + E) \log V)$ — the heap operations dominate
- **Worst case:** $O((V + E) \log V)$ — every edge causes a heap push
- **Dense graph:** $O(V^2)$ without heap is better than $O((V+E) \log V) = O(V^2 \log V)$ with heap

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Standard SSSP	"Shortest path from source to all nodes", non-negative weights	Standard Dijkstra with adjacency list and min-heap	Network Delay Time, Path with Maximum Probability
Dijkstra on Grid	2D grid with movement costs, find min cost path	Each cell is a node; use priority queue; relax neighbors	Path with Minimum Effort, Minimum Cost to Make at Least One Valid Path
Dijkstra with State	Need to track additional state (fuel, time, obstacles)	<code>dist[node][state]</code> 2D distance array; push (dist, node, state)	Cheapest Flights Within K Stops, Minimum Cost to Reach Destination with K Changes
Second Shortest Path	"Find second shortest path" or "nearly shortest path"	Run Dijkstra, then modify (remove edges, or use A* with k-shortest paths)	Second Shortest Path (CF 1005E)

Pattern	How to Identify	Approach	Example Problems
Multi-source Dijkstra	Multiple starting points (e.g., nearest restaurant)	Push all sources with <code>dist=0</code> initially	As Far from Land as Possible, Shortest Distance to a Character
Dijkstra + DP	Need to count number of shortest paths	Maintain <code>ways[v]</code> alongside <code>dist[v]</code> ; update ways when relaxing	Number of Ways to Arrive at Destination

13. Common Mistakes

- **Using Dijkstra on negative weights** — Dijkstra greedily assumes once a node is popped, its distance is final. A negative edge could create a shorter path later. This **will produce wrong answers**.
- **Not using lazy deletion** — Without lazy deletion, stale entries in the heap can cause incorrect processing or infinite loops.
- **Using `visited` array instead of lazy deletion** — Dijkstra needs to relax edges even for visited nodes if a shorter path is found. Lazy deletion is the correct approach.
- **Forgetting `long long`** — Edge weights can be large (up to 10^9), and distances can overflow 32-bit int. Use `long long` (C++) or `int64/float('inf')` (Python).
- **Incorrect INF value** — `INT_MAX` + anything overflows. Use `1e18` or `LLONG_MAX/2`.
- **Priority queue as max-heap** — In C++, `priority_queue<int>` is a max-heap. Use `priority_queue<pii, vector<pii>, greater<pii>>` for min-heap.
- **Not skipping self-loops** — Self-loops with positive weight always increase distance; skip them.
- **Not handling disconnected graphs** — Unreachable nodes will have `dist = INF`; check before using.

14. Edge Cases

- **Source = destination** → distance = 0
- **Single node** → distance = 0, no edges to process
- **Disconnected graph** → unreachable nodes remain INF
- **Graph with zero-weight edges** → Dijkstra works fine (0-1 BFS is faster but Dijkstra handles it)
- **Graph with multiple edges between same nodes** → Dijkstra handles it; the shortest edge will be picked
- **Large weights** → use `long long` / `int64`
- **Dense graph** → consider $O(V^2)$ Dijkstra without heap
- **Graph with self-loops** → skip or process (they are never beneficial for shortest paths)

15. Variations

Variation	What Changes	When Used	Importance
Dijkstra ($O(V^2)$ version)	No heap; find min dist node by linear scan each iteration	Dense graphs where $E \approx V^2$	High — useful for small V or dense graphs
Dial's Algorithm	Uses bucket queue instead of heap; $O(V + E + \text{maxWeight})$	When edge weights are small integers	Medium — niche but useful

Variation	What Changes	When Used	Importance
Dijkstra + Potentials	For Johnson's algorithm; reweight edges to avoid negatives	All-pairs shortest path with negative weights	High — for Johnson's algorithm
Dijkstra with Path Counting	Maintain <code>ways[v]</code> count of shortest paths to v	Problems asking "number of shortest paths"	Medium — common LeetCode problem
Bidirectional Dijkstra	Run Dijkstra from both source and destination simultaneously	When both source and target are known	High — interview favorite, reduces search space significantly

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
BFS	Unweighted version of Dijkstra	Use BFS for unweighted graphs ($O(V+E)$ vs $O((V+E) \log V)$)
Bellman-Ford	Handles negative weights	Use when graph has negative edges (but no negative cycles)
0-1 BFS	Dijkstra for weights $\{0, 1\}$	Faster $O(V+E)$ for 0-1 weights
Floyd-Warshall	All-pairs shortest path	Use for dense graphs, small V, or need distances between all pairs
A*	Dijkstra with heuristic	Use when you have a good heuristic to guide search toward target
Prim's Algorithm	Also uses min-heap	Prim finds MST, not shortest paths; similar structure but different goal

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Network Delay Time	LeetCode 743	Standard Dijkstra, find max distance to any node	Easy
Path With Minimum Effort	LeetCode 1631	Dijkstra on grid, minimize max edge weight	Easy

Medium

Problem	Platform	Pattern	Difficulty
Cheapest Flights Within K Stops	LeetCode 787	Dijkstra with state (k stops remaining)	Medium
Number of Ways to Arrive at Destination	LeetCode 1976	Dijkstra with path counting	Medium
Minimum Cost to Make at Least One Valid Path in a Grid	LeetCode 1368	0-1 BFS / Dijkstra on grid with direction costs	Medium

Hard

Problem	Platform	Pattern	Difficulty
Minimum Cost to Reach Destination with K Changes	LeetCode (custom)	Dijkstra with state (layered graph)	Hard
Shortest Path to Get All Keys	LeetCode 864	Dijkstra/BFS with bitmask state	Hard
Escape the Spreading Fire	LeetCode (weekly contest)	Multi-source Dijkstra + BFS	Hard

18. Interview Explanation

"Dijkstra's algorithm finds the shortest path from a single source to all nodes in a graph with non-negative edge weights. It uses a greedy approach: at each step, we pick the unvisited node with the smallest known distance from the source. We relax all its outgoing edges — if going through this node gives a shorter path to a neighbor, we update the neighbor's distance. We use a priority queue (min-heap) to efficiently get the next node with minimum distance. The algorithm has $O((V + E) \log V)$ time complexity with a binary heap. It cannot handle negative edge weights because the greedy assumption that the popped node's distance is final fails when negative edges exist. For path reconstruction, we maintain a parent array."

19. Revision Notes

- **Key idea:** Greedy — pick the closest unvisited node, relax its edges
- **Assumption:** All edge weights must be non-negative
- **Data structure:** Min-heap (priority queue) of `(distance, node)`
- **Lazy deletion:** Skip popped entries where `d != dist[u]`
- **Time:** $O((V + E) \log V)$
- **Space:** $O(V)$
- **Dense graph:** $O(V^2)$ version is faster
- **Path reconstruction:** `parent[]` array, trace back from dest
- **Common trap:** Using `visited` boolean instead of lazy deletion
- **Overflow:** Use `long long` with `INF = 1e18`
- **Negative weights:** Use Bellman-Ford

20. Final Cheat Sheet

DIJKSTRA'S ALGORITHM

WHEN: Single-source shortest path, non-negative weights

COMPLEXITY: $O((V + E) \log V)$ time, $O(V)$ space

DATA STRUCTURE: Min-heap priority queue

TEMPLATE:

```
dist[src] = 0
pq.push({0, src})
while (!pq.empty()):
    auto [d, u] = pq.top(); pq.pop()
    if (d != dist[u]) continue;           // lazy deletion
    for (auto [v, w] : adj[u]):
        if (dist[u] + w < dist[v]):
            dist[v] = dist[u] + w
            parent[v] = u
            pq.push({dist[v], v})
```

KEY RULE: Skip stale entries; relax all edges

FAILS WITH: Negative edge weights

PATH: parent[] → reverse

Bellman-Ford Algorithm

1. Overview

Bellman-Ford is a **single-source shortest path** algorithm that works with **negative edge weights** and can **detect negative cycles**. Unlike Dijkstra, it does not use a greedy approach — it iteratively relaxes all edges $|V| - 1$ times, and then checks for negative cycles with one more pass.

2. Intuition

Imagine you are spreading a rumor in a network. Each person hears the rumor and passes it to their neighbors. In the first round, you tell your immediate neighbors. In the second round, they tell their neighbors (so people 2 hops away hear it). In the third round, people 3 hops away hear it. In a graph with V nodes, the shortest path can have at most $V - 1$ edges (otherwise there would be a cycle). So after $V - 1$ rounds of spreading information, everyone has the shortest possible rumor (distance).

Now, if after $V - 1$ rounds you can still improve some distance, it means there's a **negative cycle** — a cycle whose total weight is negative, allowing you to go around it forever and keep reducing the distance.

Why Bellman-Ford works: The shortest path in a graph with no negative cycles has at most $V - 1$ edges. By relaxing all edges $V - 1$ times, we guarantee that the shortest path with up to k edges is found by the k -th iteration. This is essentially dynamic programming: $\text{dist}_k[v] = \min(\text{dist}_{k-1}[v], \text{dist}_{k-1}[u] + w(u, v))$.

3. When to Use It

- **Graph has negative edge weights** (Dijkstra would fail)
- **Need to detect negative cycles** in a graph (e.g., currency arbitrage)
- **Single-source shortest path** with constraints on number of edges (like "at most k stops")
- **Small graphs** where the $O(V \cdot E)$ complexity is acceptable
- **When you need to find shortest paths with limited number of edges** (k-th iteration gives shortest paths with $\leq k$ edges)

Trigger phrases: "negative weights", "negative cycle detection", "arbitrage", "shortest path with at most k edges", "Bellman-Ford", "graph with negative edges"

4. When Not to Use It

- **Graph has no negative edges** — Dijkstra is faster ($O((V+E) \log V)$ vs $O(V \cdot E)$)
- **Large graph** — $O(V \cdot E)$ is too slow for $V, E > 10^4$
- **Unweighted graph** — BFS is $O(V+E)$
- **Only need to detect negative cycles reachable from a specific node** — SPFA (queue-based Bellman-Ford) is often faster in practice
- **Dense graph** — For dense graphs, $O(V \cdot E) = O(V^3)$, same as Floyd-Warshall but Floyd-Warshall gives all-pairs distances

5. Core Concepts

5.1 Relaxation (Edge)

The same operation as Dijkstra: if $\text{dist}[u] + w < \text{dist}[v]$, update $\text{dist}[v] = \text{dist}[u] + w$. In Bellman-Ford, we relax **all edges** in each iteration.

5.2 $V - 1$ Iterations

The key insight: the longest possible shortest path (without cycles) has at most $V - 1$ edges. After iteration k , we have found shortest paths using at most k edges.

5.3 Negative Cycle Detection

After $V - 1$ iterations, if any edge can still be relaxed, a negative cycle exists. This is because a negative cycle allows infinite reduction in distance.

5.4 Negative Cycle Reachability

A negative cycle only matters if it is reachable from the source. If it's in a disconnected component, it doesn't affect distances from the source.

5.5 SPFA (Shortest Path Faster Algorithm)

A queue-based optimization of Bellman-Ford. Instead of relaxing all edges $V - 1$ times, only relax edges of nodes whose distance has recently changed. Average case is $O(E)$, worst case is still $O(V \cdot E)$.

6. Step-by-Step Algorithm

Input: Graph $G(V, E)$ with edge weights (can be negative), source s
 Output: Shortest distance from s to all nodes, or negative cycle detection

1. Initialize:
 - $\text{dist}[v] = \text{INF}$ for all v , $\text{dist}[s] = 0$
 - $\text{parent}[v] = -1$ for all v
2. Repeat $V - 1$ times:
 - a. For each edge (u, v, w) in the graph:
 - If $\text{dist}[u] \neq \text{INF}$ and $\text{dist}[u] + w < \text{dist}[v]$:
 - $\text{dist}[v] = \text{dist}[u] + w$
 - $\text{parent}[v] = u$
3. Check for negative cycles:

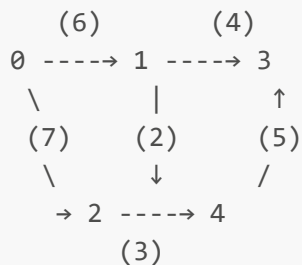
For each edge (u, v, w) :

If $\text{dist}[u] \neq \text{INF}$ and $\text{dist}[u] + w < \text{dist}[v]$:

→ Negative cycle exists (reachable from source)
4. If no negative cycle, $\text{dist}[]$ contains shortest distances.
 For unreachable nodes, $\text{dist}[v] = \text{INF}$.

7. Dry Run

Graph:



Edges (directed): $0 \rightarrow 1(6)$, $0 \rightarrow 2(7)$, $1 \rightarrow 2(2)$, $1 \rightarrow 3(4)$, $2 \rightarrow 4(3)$, $4 \rightarrow 3(5)$

Source = 0

Iteration	Edge Relaxed	dist[0]	dist[1]	dist[2]	dist[3]	dist[4]
Init	—	0	INF	INF	INF	INF
1	$0 \rightarrow 1(6)$	0	6	INF	INF	INF
1	$0 \rightarrow 2(7)$	0	6	7	INF	INF
1	$1 \rightarrow 2(2)$	0	6	6 (via 1)	INF	INF
1	$1 \rightarrow 3(4)$	0	6	6	10	INF
1	$2 \rightarrow 4(3)$	0	6	6	10	9

Iteration	Edge Relaxed	dist[0]	dist[1]	dist[2]	dist[3]	dist[4]
1	4→3(5)	0	6	6	10	9
2	1→2(2)	0	6	6	10	9
2	4→3(5)	0	6	6	9 (via 4)	9
3	4→3(5)	0	6	6	9	9
(No changes in iteration 3 and 4)		0	6	6	9	9

Final distances: [0, 6, 6, 9, 9]

Negative cycle check: No edge can be relaxed further → no negative cycle.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const ll INF = 1e18;

struct Edge {
    int u, v;
    ll w;
};

// Bellman-Ford Algorithm
// Returns shortest distances from source
// Throws if negative cycle reachable from source is detected

vector<ll> bellman_ford(int n, const vector<Edge>& edges, int src, vector<int>& parent) {
    vector<ll> dist(n, INF);
    parent.assign(n, -1);
    dist[src] = 0;

    // Relax all edges V - 1 times
    for (int i = 0; i < n - 1; i++) {
        bool updated = false;
        for (const auto& [u, v, w] : edges) {
            if (dist[u] != INF && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                parent[v] = u;
                updated = true;
            }
        }
        if (!updated) break;
    }

    // Early exit: if no update in this iteration, we're done
    if (!updated) break;
}
```

```

// Check for negative cycle reachable from source
for (const auto& [u, v, w] : edges) {
    if (dist[u] != INF && dist[u] + w < dist[v]) {
        // Negative cycle detected
        throw runtime_error("Negative cycle detected");
    }
}

return dist;
}

// Reconstruct path
vector<int> get_path(const vector<int>& parent, int dest) {
    vector<int> path;
    int cur = dest;
    while (cur != -1) {
        path.push_back(cur);
        cur = parent[cur];
    }
    reverse(path.begin(), path.end());
    return path;
}

// Example usage
int main() {
    int n = 5;
    vector<Edge> edges = {
        {0, 1, 6},
        {0, 2, 7},
        {1, 2, 2},
        {1, 3, 4},
        {2, 4, 3},
        {4, 3, 5}
    };

    int src = 0;
    vector<int> parent;

    try {
        vector<ll> dist = bellman_ford(n, edges, src, parent);

        cout << "Shortest distances from node " << src << ":\n";
        for (int i = 0; i < n; i++) {
            if (dist[i] == INF) cout << " To " << i << ": INF\n";
            else cout << " To " << i << ": " << dist[i] << "\n";
        }

        // Path to node 3
        vector<int> path = get_path(parent, 3);
        cout << "\nPath to 3: ";
        for (int v : path) cout << v << " ";
        cout << "\n";
    } catch (const exception& e) {
        cout << "Error: " << e.what() << "\n";
    }
}

```

```

    }

    return 0;
}

```

9. Python Implementation

```

from typing import List, Tuple

INF = 10**18

def bellman_ford(n: int, edges: List[Tuple[int, int, int]], src: int):
    dist = [INF] * n
    parent = [-1] * n
    dist[src] = 0

    # Relax all edges V - 1 times
    for i in range(n - 1):
        updated = False
        for u, v, w in edges:
            if dist[u] != INF and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                parent[v] = u
                updated = True
        if not updated:
            break

    # Check for negative cycle
    for u, v, w in edges:
        if dist[u] != INF and dist[u] + w < dist[v]:
            raise ValueError("Negative cycle detected")

    return dist, parent

def get_path(parent: List[int], dest: int) -> List[int]:
    path = []
    cur = dest
    while cur != -1:
        path.append(cur)
        cur = parent[cur]
    path.reverse()
    return path

# Example usage
if __name__ == "__main__":
    n = 5
    edges = [
        (0, 1, 6),

```

```

        (0, 2, 7),
        (1, 2, 2),
        (1, 3, 4),
        (2, 4, 3),
        (4, 3, 5)
    ]

    src = 0
    try:
        dist, parent = bellman_ford(n, edges, src)
        print(f"Shortest distances from node {src}:")
        for i in range(n):
            if dist[i] == INF:
                print(f"  To {i}: INF")
            else:
                print(f"  To {i}: {dist[i]}")

        path = get_path(parent, 3)
        print(f"\nPath to 3: {' '.join(map(str, path))}")
    except ValueError as e:
        print(f"Error: {e}")

```

10. Code Explanation

- **Edge list representation:** Bellman-Ford works on a list of edges (not adjacency list). This is because we need to iterate over all edges in each iteration.
- **V - 1 iterations:** The outer loop runs exactly $V - 1$ times. After iteration k , we have found shortest paths using at most k edges.
- **Early exit:** If no distance is updated in an iteration, we can stop early — all shortest paths have been found.
- **Negative cycle detection:** After $V - 1$ iterations, if any edge can still be relaxed, a negative cycle exists. This is because $V - 1$ iterations are enough to find all shortest paths in a graph without negative cycles.
- **INF handling:** We check `dist[u] != INF` before relaxing to avoid overflow (adding w to `INF`).
- **Exception handling:** We throw an exception when a negative cycle is detected.

11. Complexity Analysis

Operation	Complexity
Time Complexity	$O(V \cdot E)$ — $V - 1$ iterations, each iterating over all E edges
Space Complexity	$O(V)$ — for <code>dist</code> and <code>parent</code> arrays
Negative Cycle Check	$O(E)$ — one more pass over all edges
SPFA (average case)	$O(E)$ — but worst case is $O(V \cdot E)$

- **Best case:** $O(E)$ — if early exit happens after first few iterations
- **Worst case:** $O(V \cdot E)$ — when the graph requires all $V - 1$ iterations
- **Dense graph:** $O(V^3)$ — when $E = O(V^2)$

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Negative cycle detection	"Detect negative cycle", "arbitrage", "currency exchange"	Run Bellman-Ford, check V-th iteration for relaxation	Arbitrage (maximum profit path), Currency Exchange
Shortest path with at most k edges	"At most k stops/flights", "limited number of edges"	Stop after k iterations (not V-1)	Cheapest Flights Within K Stops
Difference constraints	"System of inequalities", " $x_i - x_j \leq c$ "	Model as graph with edges $j \rightarrow i$ with weight c ; Bellman-Ford finds feasible solution	Layout (POJ 3169), Scheduling with constraints
Minimum cost flow	Flow problems with negative costs	Bellman-Ford used in successive shortest augmenting path algorithm	Min Cost Max Flow
SPFA optimization	Large graphs, need faster Bellman-Ford in practice	Use queue, only relax edges of nodes whose distance changed	Various CF problems with negative weights

13. Common Mistakes

- **Not using `dist[u] != INF` check** — Adding `w` to `INF` causes overflow and wraps around to a small number, leading to incorrect results.
- **Running `V` iterations instead of `V - 1`** — The `V`-th iteration is for negative cycle detection, not for relaxation. If you run `V` iterations for relaxation, you might get wrong distances.
- **Not detecting negative cycles correctly** — A negative cycle only matters if it's reachable from the source. Graph may have a negative cycle in an unreachable component.
- **Forgetting directed vs undirected** — Bellman-Ford works on directed graphs. For undirected graphs, add both directed edges.
- **Using adjacency list** — Bellman-Ford needs to iterate over all edges; using adjacency list makes this harder. Use edge list.
- **Incorrect early exit** — If early exit happens prematurely, we might miss shorter paths. Early exit is safe only if no updates happen in an entire iteration.
- **Overflow with `INT_MAX`** — Use `LLONG_MAX / 2` or `1e18` as `INF`.
- **Not handling the case where negative cycle is on the path to destination** — If a negative cycle is reachable, the distance to some nodes is undefined (can be arbitrarily negative).

14. Edge Cases

- **Empty graph (`V=0`, `E=0`)** → no edges to process, only source exists
- **Single node** → distance = 0, no edges
- **Graph with a negative cycle reachable from source** → throw error / report detection
- **Graph with a negative cycle NOT reachable from source** → distances are still valid for reachable nodes

- **All edges have negative weights** → Bellman-Ford works correctly
- **Source is in a disconnected component** → distances to other components remain INF
- **Graph with self-loop negative edge** → negative cycle detected immediately
- **Multiple edges between same nodes** → all edges are considered

15. Variations

Variation	What Changes	When Used	Importance
SPFA (Queue-based)	Use queue to track nodes whose distance changed; only relax their outgoing edges	Faster in practice (average $O(E)$), worst case $O(V \cdot E)$	High — often used in CP
Bellman-Ford with k iterations	Stop after k iterations instead of $V-1$	When limit on number of edges in path is given	High — LeetCode 787 pattern
Bellman-Ford for negative cycle detection only	Initialize all dist to 0, run V iterations	Detect any negative cycle in the graph (not just from source)	Medium — useful for arbitrage
Bellman-Ford for difference constraints	Each constraint $x_i - x_j \leq c$ becomes edge $j \rightarrow i$ with weight c	System of linear inequalities, scheduling	Medium — important for certain CP problems

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Dijkstra	Also SSSP but no negative weights	Use Dijkstra when no negative edges (faster)
Floyd-Warshall	All-pairs, handles negative weights	Use for dense graphs, small V , need all-pairs
SPFA	Optimization of Bellman-Ford	Use in practice for faster Bellman-Ford
Johnson's Algorithm	Uses Bellman-Ford + Dijkstra for all-pairs	Use when graph has negative weights but is sparse
Topological Sort DP	Shortest path in DAG ($O(V+E)$)	Use when graph is a DAG (much faster)

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Bellman-Ford Implementation	GFG	Standard Bellman-Ford implementation	Easy
Negative Weight Cycle	GFG	Detect negative cycle	Easy

Medium

Problem	Platform	Pattern	Difficulty
Cheapest Flights Within K Stops	LeetCode 787	Bellman-Ford with k iterations	Medium
Arbitrage	GFG	Negative cycle detection (log transformation)	Medium
Layout (POJ 3169)	POJ	Difference constraints with Bellman-Ford	Medium

Hard

Problem	Platform	Pattern	Difficulty
Minimum Cost to Reach Destination with K Stops	GFG	Bellman-Ford with state	Hard
Currency Exchange	GFG	Arbitrage detection with Bellman-Ford	Hard
Shortest Path with Negative Weights	CodeChef	Bellman-Ford with negative edges	Hard

18. Interview Explanation

"Bellman-Ford finds the shortest paths from a single source in a graph that may have negative edge weights. It works by relaxing all edges $V - 1$ times. The key insight is that the shortest path without cycles has at most $V - 1$ edges, so after $V - 1$ rounds of relaxing all edges, we have the shortest distances. Then, we do one more pass to check for negative cycles — if any edge can still be relaxed, a negative cycle exists. The complexity is $O(V \cdot E)$. While Dijkstra is faster, Bellman-Ford handles negative weights and detects negative cycles. It's also useful for problems with limited number of edges (like 'at most k stops') and for difference constraints."

19. Revision Notes

- **Key idea:** Relax all edges $V - 1$ times; shortest path has $\leq V - 1$ edges
- **V-th iteration:** Check for negative cycles (if any edge relaxes $\rightarrow$ negative cycle)
- **Time:** $O(V \cdot E)$
- **Space:** $O(V)$
- **Data structure:** Edge list (list of (u, v, w) tuples)
- **Early exit:** If no updates in an iteration, stop
- **INF check:** Always check `dist[u] != INF` before relaxing
- **Negative cycle:** Only matters if reachable from source
- **k-stops:** Stop after k iterations
- **SPFA:** Queue-based optimization, average $O(E)$, worst $O(V \cdot E)$

20. Final Cheat Sheet

BELLMAN-FORD ALGORITHM

WHEN: Negative weights, negative cycle detection, k-edge limit

COMPLEXITY: $O(V \cdot E)$ time, $O(V)$ space

DATA STRUCTURE: Edge list

TEMPLATE:

```
dist[src] = 0
for (i = 0; i < n-1; i++):
    for (u, v, w) in edges:
        if (dist[u] != INF && dist[u] + w < dist[v]):
            dist[v] = dist[u] + w
            parent[v] = u
```

```
// Negative cycle check
for (u, v, w) in edges:
    if (dist[u] != INF && dist[u] + w < dist[v]):
        → NEGATIVE CYCLE
```

KEY: $V-1$ relaxations, then one more pass for cycle detection

FAILS WITH: Nothing (handles all cases)

PATH: `parent[]` → reverse

Floyd-Warshall Algorithm

1. Overview

Floyd-Warshall is an **all-pairs shortest path** algorithm. Unlike Dijkstra and Bellman-Ford (which find paths from one source), Floyd-Warshall finds the shortest path between **every pair of nodes** in a single run. It works with **positive and negative edge weights** (but no negative cycles) and uses **dynamic programming**.

2. Intuition

Imagine you have a map of cities and you want to find the shortest distance between every pair of cities. You could run Dijkstra from each city, but that's slow. Instead, consider this: the shortest path from city A to city B either goes through city C or doesn't. If it goes through C, you can combine the shortest path from A to C with the shortest path from C to B.

Floyd-Warshall works by considering one "intermediate" node at a time. For each pair (i, j) , we ask: "Is the path from i to j through k shorter than the current known path?" We do this for all k from 0 to $V-1$.

Why it works: The algorithm maintains a 2D distance matrix `dist[i][j]` which is the shortest known distance from i to j using only nodes $\{0, 1, \dots, k-1\}$ as intermediates. When we add node k as a possible intermediate, we update: `dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])`. After processing all nodes as intermediates, we have the true shortest paths.

3. When to Use It

- **All-pairs shortest path** needed
- **Graph is dense** ($E \approx V^2$ — Floyd-Warshall's $O(V^3)$ is optimal)
- **Small V** (typically $V \leq 500$, sometimes up to 1000 with optimization)
- **Graph has negative edge weights** (but no negative cycles)
- **Need to detect reachability** (transitive closure) — use Floyd-Warshall with boolean matrix
- **Need shortest path in a graph with small number of nodes** ($V \leq 400$ is common in CP)

Trigger phrases: "all-pairs shortest path", "shortest distance between every pair", "transitive closure", "reachability matrix", "minimize over all pairs", "small graph"

4. When Not to Use It

- **Large V** — $O(V^3)$ is too slow for $V > 1000$ (use Johnson's algorithm or run Dijkstra from each node)
- **Sparse graph** — Running Dijkstra V times is $O(V(E + V \log V))$ which is faster for sparse graphs
- **Single-source shortest path** — Use Dijkstra or Bellman-Ford ($O(V^2)$ or $O((V+E)\log V)$ vs $O(V^3)$)
- **Graph has negative cycles** — Floyd-Warshall can detect them but cannot compute shortest paths
- **Memory is tight** — $O(V^2)$ memory for distance matrix; for $V = 10^5$, this is impossible
- **Need to reconstruct paths** — Path reconstruction requires $O(V^3)$ extra space for the next matrix (or compute on the fly)

5. Core Concepts

5.1 Dynamic Programming Formulation

`dist[k][i][j]` = shortest path from i to j using only nodes $\{0, 1, \dots, k-1\}$ as intermediates. In practice, we reuse the same 2D matrix and update in-place.

5.2 Optimal Substructure

The shortest path from i to j either goes through k or doesn't:

- If it doesn't go through k : `dist[i][j]` stays the same
- If it goes through k : `dist[i][j] = dist[i][k] + dist[k][j]`

5.3 In-Place Update

Since we update `dist[i][j]` using `dist[i][k]` and `dist[k][j]`, and `dist[i][k]` and `dist[k][j]` are already finalized (since k is the new intermediate), we can update in-place without needing a separate 3D array.

5.4 Transitive Closure

A boolean version of Floyd-Warshall: `reachable[i][j] = reachable[i][j] || (reachable[i][k] && reachable[k][j])`. This computes which nodes can reach which other nodes.

5.5 Negative Cycle Detection

After running Floyd-Warshall, if `dist[i][i] < 0` for any i , a negative cycle exists (since the distance from a node to itself should be 0).

6. Step-by-Step Algorithm

Input: Graph $G(V, E)$ with adjacency matrix

Output: $\text{dist}[i][j]$ = shortest distance from i to j for all pairs

1. Initialize dist matrix:

- $\text{dist}[i][j] = 0$ if $i == j$
- $\text{dist}[i][j] = w(i, j)$ if edge exists
- $\text{dist}[i][j] = \text{INF}$ otherwise

2. For $k = 0$ to $V-1$: // intermediate node

For $i = 0$ to $V-1$: // source node

For $j = 0$ to $V-1$: // destination node

If $\text{dist}[i][k] \neq \text{INF}$ and $\text{dist}[k][j] \neq \text{INF}$:

$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$

3. Check for negative cycles:

For $i = 0$ to $V-1$:

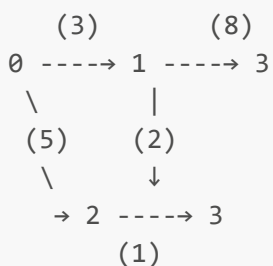
If $\text{dist}[i][i] < 0$: $\rightarrow$ negative cycle exists

4. $\text{dist}[i][j]$ is the shortest distance from i to j .

If $\text{dist}[i][j] == \text{INF}$, nodes are unreachable.

7. Dry Run

Graph:



Graph (directed): $0 \rightarrow 1(3)$, $0 \rightarrow 2(5)$, $1 \rightarrow 2(2)$, $1 \rightarrow 3(8)$, $2 \rightarrow 3(1)$

Initial distance matrix:

$i \setminus j$	0	1	2	3
0	0	3	5	INF
1	INF	0	2	8
2	INF	INF	0	1
3	INF	INF	INF	0

k = 0: Consider node 0 as intermediate

- $\text{dist}[1][2] = \min(2, \text{dist}[1][0] + \text{dist}[0][2]) = \min(2, \text{INF}) = 2$
- $\text{dist}[1][3] = \min(8, \text{dist}[1][0] + \text{dist}[0][3]) = \min(8, \text{INF}) = 8$
- No changes (node 0 cannot reach others via 1 or 2)

k = 1: Consider node 1 as intermediate

- $\text{dist}[0][2] = \min(5, \text{dist}[0][1] + \text{dist}[1][2]) = \min(5, 3+2=5) = 5$
- $\text{dist}[0][3] = \min(\text{INF}, \text{dist}[0][1] + \text{dist}[1][3]) = \min(\text{INF}, 3+8=11) = \mathbf{11}$
- $\text{dist}[2][3] = \min(1, \text{dist}[2][1] + \text{dist}[1][3]) = \min(1, \text{INF}) = 1$

k = 2: Consider node 2 as intermediate

- $\text{dist}[0][3] = \min(11, \text{dist}[0][2] + \text{dist}[2][3]) = \min(11, 5+1=6) = \mathbf{6}$
- $\text{dist}[1][3] = \min(8, \text{dist}[1][2] + \text{dist}[2][3]) = \min(8, 2+1=3) = \mathbf{3}$

k = 3: Consider node 3 as intermediate

- No changes (node 3 has no outgoing edges)

Final distance matrix:

i\j	0	1	2	3
0	0	3	5	6
1	INF	0	2	3
2	INF	INF	0	1
3	INF	INF	INF	0

Interpretation: $\text{dist}[0][3] = 6$ (path: $0 \rightarrow 2 \rightarrow 3$ or $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$), $\text{dist}[1][3] = 3$ (path: $1 \rightarrow 2 \rightarrow 3$)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const ll INF = 1e18;

// Floyd-Warshall All-Pairs Shortest Path
// Works with negative weights (no negative cycles)
// Returns a 2D distance matrix

vector<vector<ll>> floyd_warshall(const vector<vector<ll>>& adj_matrix) {
    int n = adj_matrix.size();
    vector<vector<ll>> dist = adj_matrix; // Copy

    // Initialize diagonal to 0 and INF for missing edges
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) dist[i][j] = 0;
        }
    }
}
```

```

        if (dist[i][j] == 0 && i != j) dist[i][j] = INF; // 0 means no edge
    }
}

// Main DP loop
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        if (dist[i][k] == INF) continue; // Optimization
        for (int j = 0; j < n; j++) {
            if (dist[k][j] == INF) continue; // Optimization
            if (dist[i][k] + dist[k][j] < dist[i][j]) {
                dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }
}

// Check for negative cycles
for (int i = 0; i < n; i++) {
    if (dist[i][i] < 0) {
        // Negative cycle detected
        // Mark all nodes affected by negative cycles
        // (In practice, throw or return empty)
    }
}

return dist;
}

// Example usage
int main() {
    int n = 4;
    // Initialize with 0 (meaning no edge for non-diagonal)
    vector<vector<ll>> adj_matrix(n, vector<ll>(n, 0));

    // Set edges: adj_matrix[u][v] = w
    adj_matrix[0][1] = 3;
    adj_matrix[0][2] = 5;
    adj_matrix[1][2] = 2;
    adj_matrix[1][3] = 8;
    adj_matrix[2][3] = 1;

    vector<vector<ll>> dist = floyd_warshall(adj_matrix);

    cout << "All-Pairs Shortest Distances:\n";
    cout << "  ";
    for (int j = 0; j < n; j++) cout << setw(4) << j;
    cout << "\n";
    for (int i = 0; i < n; i++) {
        cout << i << ": ";
        for (int j = 0; j < n; j++) {
            if (dist[i][j] >= INF/2) cout << "  INF";
            else cout << setw(4) << dist[i][j];
        }
    }
}

```

```

        cout << "\n";
    }

    return 0;
}

```

9. Python Implementation

```

from typing import List

INF = 10**18

def floyd_warshall(adj_matrix: List[List[int]]):
    n = len(adj_matrix)
    dist = [[0] * n for _ in range(n)]

    # Initialize distance matrix
    for i in range(n):
        for j in range(n):
            if i == j:
                dist[i][j] = 0
            elif adj_matrix[i][j] != 0:
                dist[i][j] = adj_matrix[i][j]
            else:
                dist[i][j] = INF

    # Main DP loop
    for k in range(n):
        for i in range(n):
            if dist[i][k] == INF:
                continue
            for j in range(n):
                if dist[k][j] == INF:
                    continue
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]

    # Check for negative cycles
    for i in range(n):
        if dist[i][i] < 0:
            print("Warning: Negative cycle detected")
            break

    return dist

# Example usage
if __name__ == "__main__":
    n = 4
    # 0 means no edge (for non-diagonal)

```

```

adj_matrix = [[0] * n for _ in range(n)]
adj_matrix[0][1] = 3
adj_matrix[0][2] = 5
adj_matrix[1][2] = 2
adj_matrix[1][3] = 8
adj_matrix[2][3] = 1

dist = floyd_warshall(adj_matrix)

print("All-Pairs Shortest Distances:")
print(" ", end=" ")
for j in range(n):
    print(f"{j:4}", end=" ")
print()
for i in range(n):
    print(f"{i}: ", end=" ")
    for j in range(n):
        if dist[i][j] >= INF // 2:
            print(" INF", end=" ")
        else:
            print(f"{dist[i][j]:4}", end=" ")
    print()

```

10. Code Explanation

- **Initialization:** The adjacency matrix is copied. We set `dist[i][i] = 0` and `dist[i][j] = INF` if there's no edge. We use 0 to mean "no edge" in the input (since edges can have positive weights).
- **Triple nested loop:** The outer loop `k` is the intermediate node. The inner loops `i` and `j` are source and destination. For each pair (i, j) , we check if going through `k` gives a shorter path.
- **Optimization:** Skip if `dist[i][k] == INF` or `dist[k][j] == INF` — no point in checking.
- **In-place update:** We update `dist[i][j]` in-place. This is valid because `dist[i][k]` and `dist[k][j]` won't change during this iteration (they use only nodes $\{0 \dots k-1\}$ as intermediates, and we haven't changed them yet).
- **Negative cycle detection:** After the algorithm, if `dist[i][i] < 0`, there's a negative cycle (since `dist[i][i]` should be 0 for shortest paths).

11. Complexity Analysis

Operation	Complexity
Time Complexity	$O(V^3)$ — three nested loops
Space Complexity	$O(V^2)$ — for the distance matrix
Path Reconstruction Space	$O(V^3)$ if storing <code>next[i][j]</code> matrix, or $O(V^2)$ for computing on the fly
Transitive Closure (boolean)	$O(V^3)$ — same structure, boolean operations

- **Best case:** $O(V^3)$ — always the same regardless of graph structure
- **Worst case:** $O(V^3)$ — no early exit possible
- **For $V = 500$:** ~125 million operations, which is fast in C++

- **For $V = 1000$:** ~1 billion operations, which may be borderline (use optimization or choose another algorithm)

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
All-pairs shortest path	"Shortest distance between every pair", "minimize over all pairs"	Standard Floyd-Warshall	City and Campers, Finding shortest paths
Transitive closure	"Reachability", "can reach", "is connected"	Boolean Floyd-Warshall with OR operations	Course Schedule (reachability), Transitive Closure
Minimax / Maximin path	"Minimize the maximum edge weight along path"	$\text{dist}[i][j] = \min(\text{dist}[i][j], \max(\text{dist}[i][k], \text{dist}[k][j]))$	Minimax Path, Minimize the maximum edge
Detect negative cycles	"Detect negative cycle", "arbitrage"	Check $\text{dist}[i][i] < 0$ after running	Arbitrage detection
Shortest path with at most k edges (all pairs)	"At most k edges"	Run Floyd-Warshall but restrict to k iterations	Various CP problems
Graph diameter	"Diameter of graph", "longest shortest path"	Run Floyd-Warshall, find $\max \text{dist}[i][j]$	Graph diameter, network eccentricity

13. Common Mistakes

- **Not initializing diagonal to 0** — $\text{dist}[i][i]$ must be 0. If you leave it as INF, the algorithm won't work.
- **Using 0 to mean "no edge" without proper handling** — If 0 is a valid edge weight, use a separate sentinel (like -1 or INF).
- **Forgetting that Floyd-Warshall works only for directed graphs** — For undirected graphs, add both directed edges.
- **Not checking for negative cycles** — If a negative cycle exists, distances can be arbitrarily negative and the results are meaningless.
- **Accessing out-of-bounds** — The triple loop must be $k < n, i < n, j < n$.
- **Integer overflow** — $\text{dist}[i][k] + \text{dist}[k][j]$ can overflow if both are large. Use `long long` and check for INF.
- **Using the algorithm for large V** — $O(V^3)$ is too slow for $V > 1000$. Use Johnson's algorithm instead.
- **Not handling INF correctly** — Comparison $\text{dist}[i][k] + \text{dist}[k][j] < \text{dist}[i][j]$ can be wrong if INF is `INT_MAX` (overflow). Use $\text{INF}/2$ or check for INF before adding.

14. Edge Cases

- **Single node ($V=1$)** → distance matrix is $[[0]]$

- **Empty graph (no edges)** → INF for all $i \neq j$
- **Complete graph** → all pairs have edges; Floyd-Warshall works fine
- **Graph with negative cycles** → detect and report; distances are meaningless
- **Disconnected graph** → INF for unreachable pairs
- **Graph with self-loops** → self-loop weight doesn't matter if $\text{dist}[i][i] = 0$; negative self-loop = negative cycle
- **Multiple edges between same nodes** → use the smallest weight during initialization
- **Undirected graph** → initialize both $\text{dist}[i][j]$ and $\text{dist}[j][i]$ with the same weight

15. Variations

Variation	What Changes	When Used	Importance
Transitive Closure (Warshall's Algorithm)	Boolean matrix; $\text{reachable}[i][j] \mid= \text{reachable}[i][k] \ \&\& \ \text{reachable}[k][j]$	Reachability only, no weights	High — common in CP
Minimax / Maximin Path	$\text{dist}[i][j] = \min(\text{dist}[i][j], \max(\text{dist}[i][k], \text{dist}[k][j]))$	Minimize maximum edge weight on path	Medium — useful for certain problems
Path Reconstruction	Maintain $\text{next}[i][j]$ matrix to reconstruct paths	Need actual paths, not just distances	Medium — interview extension
Floyd-Warshall with k iterations	Stop after k iterations	Shortest path with at most k edges	Medium — for k-stops problems
Detecting All Negative Cycles	Mark nodes that are part of or reachable from negative cycles	Complete analysis of graph with negative cycles	Low — advanced use

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Dijkstra (V times)	$O(V(E + V \log V))$ for all-pairs	Choose for sparse graphs ($E \approx V$)
Johnson's Algorithm	Bellman-Ford + $V \times$ Dijkstra	Choose for sparse graphs with negative weights
Bellman-Ford	SSSP with negative weights	Use when only single-source is needed
BFS (V times)	$O(V(V+E))$ for unweighted all-pairs	Use when graph is unweighted
Matrix Multiplication (min-plus)	Alternative to Floyd-Warshall, $O(V^3 \log V)$ for repeated squaring	Use when you need powers of the distance matrix (e.g., "paths with exactly k edges")

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Floyd-Warshall Implementation	GFG	Standard implementation	Easy
Find the City With the Smallest Number of Neighbors at a Threshold Distance	LeetCode 1334	Floyd-Warshall + count reachable cities	Easy

Medium

Problem	Platform	Pattern	Difficulty
Course Schedule IV	LeetCode 1462	Transitive closure (boolean Floyd-Warshall)	Medium
Minimum Cost to Make at Least One Valid Path in a Grid	LeetCode 1368	Can use Floyd-Warshall on small graphs	Medium

Hard

Problem	Platform	Pattern	Difficulty
Shortest Path Visiting All Nodes	LeetCode 847	Floyd-Warshall to precompute distances between all pairs, then DP	Hard
Escape the Spreading Fire	LeetCode (weekly)	Multi-source BFS + Floyd-Warshall for precomputation	Hard
Minimum Cost to Connect Two Cities	GFG	Floyd-Warshall with edge costs	Hard

18. Interview Explanation

"Floyd-Warshall finds the shortest path between every pair of nodes in a graph. It's a dynamic programming algorithm that uses a 2D distance matrix and considers each node as a potential intermediate point. The core idea is: for each pair (i, j), the shortest path either goes through node k or doesn't. We update $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$. We run this for all k from 0 to V-1. The time complexity is $O(V^3)$ and space is $O(V^2)$. It handles negative weights (but not negative cycles) and can also detect negative cycles by checking if $\text{dist}[i][i] < 0$ after running. It's best for dense graphs with small V. For sparse graphs, running Dijkstra from each node is more efficient."

19. Revision Notes

- **Key idea:** DP — consider each node as intermediate
- **Formula:** $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$
- **Time:** $O(V^3)$
- **Space:** $O(V^2)$
- **Handles:** Positive and negative weights, detects negative cycles
- **Does NOT handle:** Negative cycles (distances become meaningless)
- **Initialization:** $\text{dist}[i][i] = 0, \text{dist}[i][j] = w(i, j)$ if edge exists, else INF

- **Order of loops:** $k \rightarrow i \rightarrow j$ (k is the intermediate)
- **Optimization:** Skip if `dist[i][k] == INF` or `dist[k][j] == INF`
- **Transitive closure:** Boolean version, use `||` and `&&` instead of `min` and `+`
- **Path reconstruction:** Maintain `next[i][j]` matrix initialized to j

20. Final Cheat Sheet

FLOYD-WARSHALL ALGORITHM

WHEN: All-pairs shortest path, dense graph, small V (≤ 500)

COMPLEXITY: $O(V^3)$ time, $O(V^2)$ space

HANDLES: Positive and negative weights

FAILS WITH: Negative cycles

TEMPLATE:

```
// Initialize dist[V][V]
for (k = 0; k < n; k++)
    for (i = 0; i < n; i++)
        if (dist[i][k] == INF) continue;
        for (j = 0; j < n; j++)
            if (dist[k][j] == INF) continue;
            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);

// Negative cycle check
for (i = 0; i < n; i++)
    if (dist[i][i] < 0) → NEGATIVE CYCLE
```

KEY: Node k is the intermediate; process all k

PATH: Maintain `next[i][j]` matrix

0-1 BFS

1. Overview

0-1 BFS is a specialized algorithm for finding the shortest path in a graph where **all edge weights are either 0 or 1**. It is faster than Dijkstra ($O(V + E)$ vs $O((V + E) \log V)$) and handles the common case of graphs with equal-cost and zero-cost edges.

2. Intuition

Imagine you have a map where some roads are free (cost 0) and some roads cost 1 unit. You want to find the cheapest route. You can think of 0-cost edges as "free moves" — you should take them as early as possible because they don't increase your cost. 1-cost edges are "paid moves" — you take them only when necessary.

The trick is to use a **deque** (double-ended queue) instead of a regular queue or priority queue. When you pop a node, you examine its neighbors:

- If the edge has weight **0**, push the neighbor to the **front** of the deque (process it immediately, same level)
- If the edge has weight **1**, push the neighbor to the **back** of the deque (process it later, next level)

Why it works: This maintains the invariant that the deque always contains nodes sorted by distance from the source. Nodes at distance d are at the front, and nodes at distance $d+1$ are at the back. When you pop from the front, you always get the node with the smallest distance.

3. When to Use It

- **Edge weights are only 0 or 1** — this is the exact use case
- **Graphs with equal-cost and zero-cost edges** (e.g., grid with movable and destructible obstacles)
- **Shortest path in a graph where some moves are free**
- **Minimizing number of "paid" operations**
- **Grid problems where you can move in some directions for free**
- **Any graph where Dijkstra would work but weights are limited to {0, 1}**

Trigger phrases: "0-1 BFS", "weights are 0 or 1", "minimum cost with free moves", "minimize number of paid operations", "deque BFS", "dial's algorithm for 2 buckets"

4. When Not to Use It

- **Edge weights are not 0 or 1** — Dijkstra or Bellman-Ford is needed
- **All edge weights are the same** — Use regular BFS (simpler, same complexity)
- **Graph has negative weights** — 0-1 BFS doesn't handle negative weights
- **Need to find paths with more than 2 distinct cost values** — Dijkstra is the general solution
- **Graph is dense** — Both 0-1 BFS and Dijkstra have similar performance for dense graphs; Dijkstra with $O(V^2)$ might be better

5. Core Concepts

5.1 Deque (Double-Ended Queue)

The core data structure. We push 0-weight edges to the front and 1-weight edges to the back. This maintains nodes sorted by distance.

5.2 Distance Array

Same as BFS/Dijkstra. `dist[v]` stores the shortest distance from source to `v`. Initialized to `INF`, `dist[src] = 0`.

5.3 Level-by-Level with 0-Cost Skips

When a 0-weight edge is used, the neighbor is at the same distance level, so it should be processed immediately (pushed to front). This is like "skipping ahead" without increasing cost.

5.4 Invariant Preservation

The deque always contains nodes in non-decreasing order of distance. The front has the smallest distance, and the back has the largest. This is the key property that makes the algorithm work.

6. Step-by-Step Algorithm

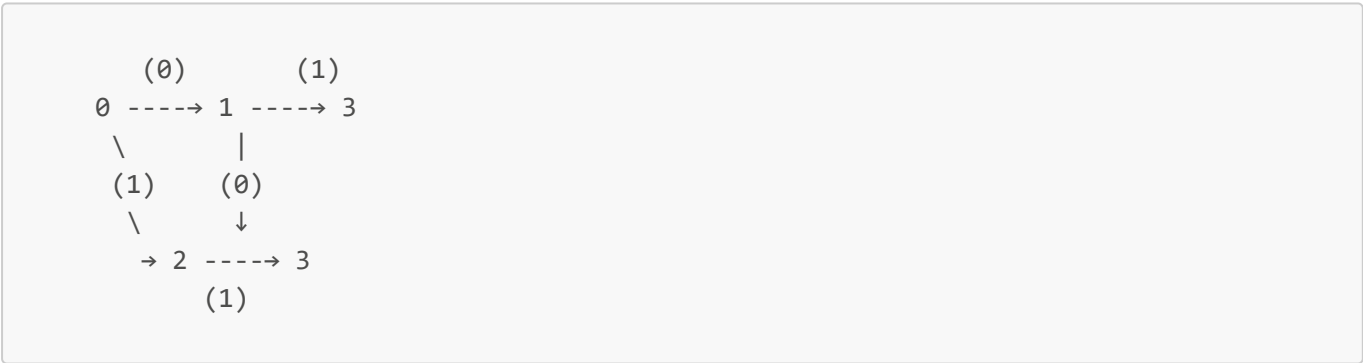
Input: Graph $G(V, E)$ with weights $\in \{0, 1\}$, source s

Output: Shortest distance from s to all nodes

1. Initialize:
 - $\text{dist}[v] = \text{INF}$ for all v , $\text{dist}[s] = 0$
 - deque dq
 - $dq.\text{push_front}(s)$
2. While dq is not empty:
 - a. $u = dq.\text{front}()$, $dq.\text{pop_front}()$
 - b. For each neighbor v of u with weight $w \in \{0, 1\}$:
 - If $\text{dist}[u] + w < \text{dist}[v]$:
 - $\text{dist}[v] = \text{dist}[u] + w$
 - If $w == 0$: $dq.\text{push_front}(v)$
 - If $w == 1$: $dq.\text{push_back}(v)$
3. $\text{dist}[]$ contains shortest distances.
If $\text{dist}[v] == \text{INF}$, v is unreachable.

7. Dry Run

Graph:



Edges: $0 \rightarrow 1(0)$, $0 \rightarrow 2(1)$, $1 \rightarrow 2(0)$, $1 \rightarrow 3(1)$, $2 \rightarrow 3(1)$

Source = 0

Step	Deque (front → back)	Pop	dist array	Neighbors processed
Init	[0]	—	[0, INF, INF, INF]	—
1	[0]	0	[0, 0, 1, INF]	1($w=0$, push_front): $\text{dist}[1]=0$; 2($w=1$, push_back): $\text{dist}[2]=1$
2	[1, 2]	1	[0, 0, 0, 1]	2($w=0$, push_front): $\text{dist}[2]=0 \rightarrow$ deque becomes [2, 2]; 3($w=1$, push_back): $\text{dist}[3]=1$

Step	Deque (front → back)	Pop	dist array	Neighbors processed
3	[2, 2, 3]	2	[0, 0, 0, 1]	3(w=1): dist[2]+1=1 ≥ dist[3]=1, no update
4	[2, 3]	2	[0, 0, 0, 1]	3(w=1): dist[2]+1=1 ≥ dist[3]=1, no update
5	[3]	3	[0, 0, 0, 1]	—

Final distances: [0, 0, 0, 1]

Paths:

- 0→1: cost 0 (direct edge with weight 0)
- 0→2: cost 0 (0→1→2 with weights 0, 0)
- 0→3: cost 1 (0→1→3 with weight 1, or 0→2→3 with weight 2)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;

// 0-1 BFS: Shortest path in graph with weights 0 or 1
// Returns vector of distances from source

vector<int> bfs_01(const vector<vector<pair<int,int>>>& adj, int src) {
    int n = adj.size();
    vector<int> dist(n, INF);
    deque<int> dq;

    dist[src] = 0;
    dq.push_front(src);

    while (!dq.empty()) {
        int u = dq.front();
        dq.pop_front();

        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                if (w == 0) {
                    dq.push_front(v);
                } else {
                    dq.push_back(v);
                }
            }
        }
    }

    return dist;
}
```

```

}

// Example usage: Grid with free and paid moves
int min_cost_grid(const vector<vector<int>>& grid, int src_r, int src_c) {
    int rows = grid.size(), cols = grid[0].size();
    int n = rows * cols;
    vector<vector<pair<int,int>>> adj(n);

    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    auto id = [&](int r, int c) { return r * cols + c; };

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            int u = id(r, c);
            for (int d = 0; d < 4; d++) {
                int nr = r + dr[d], nc = c + dc[d];
                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols) {
                    int v = id(nr, nc);
                    int w = (grid[nr][nc] == 0) ? 0 : 1; // 0 = free cell, 1 =
obstacle (paid)
                    adj[u].push_back({v, w});
                }
            }
        }
    }

    vector<int> dist = bfs_01(adj, id(src_r, src_c));
    return dist[id(rows-1, cols-1)];
}

int main() {
    // Example: graph with 0-1 weights
    int n = 5;
    vector<vector<pair<int,int>>> adj(n);

    adj[0] = {{1, 0}, {2, 1}};
    adj[1] = {{2, 0}, {3, 1}};
    adj[2] = {{3, 1}, {4, 0}};
    adj[3] = {{4, 1}};
    adj[4] = {};

    vector<int> dist = bfs_01(adj, 0);

    cout << "Shortest distances from 0:\n";
    for (int i = 0; i < n; i++) {
        if (dist[i] == INF) cout << " To " << i << ": INF\n";
        else cout << " To " << i << ": " << dist[i] << "\n";
    }

    return 0;
}

```

9. Python Implementation

```
from collections import deque
from typing import List, Tuple

INF = 10**9

def bfs_01(adj: List[List[Tuple[int, int]]], src: int) -> List[int]:
    n = len(adj)
    dist = [INF] * n
    dq = deque()

    dist[src] = 0
    dq.appendleft(src)

    while dq:
        u = dq.popleft()

        for v, w in adj[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)

    return dist

# Example usage
if __name__ == "__main__":
    n = 5
    adj = [[] for _ in range(n)]
    adj[0] = [(1, 0), (2, 1)]
    adj[1] = [(2, 0), (3, 1)]
    adj[2] = [(3, 1), (4, 0)]
    adj[3] = [(4, 1)]
    adj[4] = []

    dist = bfs_01(adj, 0)

    print("Shortest distances from 0:")
    for i in range(n):
        if dist[i] == INF:
            print(f"  To {i}: INF")
        else:
            print(f"  To {i}: {dist[i]}")
```

10. Code Explanation

- **Deque:** We use `deque<int>` (C++) or `deque` (Python). `push_front/appendleft` for 0-weight edges, `push_back/append` for 1-weight edges.
- **Distance update:** Same as Dijkstra: if `dist[u] + w < dist[v]`, update and push.
- **0-weight edge → front:** The neighbor is at the same distance level, so it should be processed before any nodes at the next level.
- **1-weight edge → back:** The neighbor is at the next distance level, so it goes to the back.
- **No visited array:** Unlike BFS, we might find a shorter path to a node later (via a different combination of 0-weight edges). We use distance comparison to decide whether to update.
- **No lazy deletion:** Unlike Dijkstra, we don't need to handle stale entries because the deque ordering ensures we always process nodes in order of increasing distance.

11. Complexity Analysis

Operation	Complexity
Time Complexity	$O(V + E)$ — each node is pushed/popped at most once per edge weight type
Space Complexity	$O(V)$ — for deque and distance array
Compared to Dijkstra	$O(V + E)$ vs $O((V + E) \log V)$ — faster by a log factor

- **Best case:** $O(V + E)$ — the whole graph is traversed
- **Worst case:** $O(V + E)$ — same as best case, no extra overhead
- **Note:** Each node can be pushed to the deque multiple times (once for each relaxation), but the total number of pushes is bounded by $O(E)$ since each edge causes at most one push.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Grid with free/paid cells	Grid with 2 types of cells; one costs 0, other costs 1	0-1 BFS on grid; 0-cost cells don't increase distance	Minimum Cost to Make at Least One Valid Path
Minimum flips/changes	"Minimum number of flips", "change direction", "modify path"	Treat unchanged path as 0-cost, changes as 1-cost	Minimum Number of Flips to Convert Binary Matrix
Graph with two edge types	Some edges free, some cost 1	0-1 BFS with deque	Shortest Path with Free and Paid Edges
Minimum obstacles to remove	"Minimize obstacles removed", "destroy walls"	Walkable cells cost 0, obstacles cost 1 (to remove)	Shortest Path in Grid with Obstacles Elimination

13. Common Mistakes

- **Using visited array** — A node may be reached again with a shorter distance via a chain of 0-weight edges. Don't skip already visited nodes; always check distance.
- **Forgetting to check `dist[u] + w < dist[v]`** — Even with the deque strategy, we need to check if we've found a shorter path.

- **Using `push_front` for 1-weight edges** — This would break the ordering and produce incorrect results.
- **Not initializing `dist` to INF** — Uninitialized distances can cause wrong comparisons.
- **Using the algorithm for non-0/1 weights** — 0-1 BFS only works for weights 0 and 1. For other small weights, use Dial's algorithm.
- **Not handling self-loops** — Self-loop with weight 0 can cause infinite loop if not handled; check `dist[u] + w < dist[v]` prevents this.
- **Forgetting `pop_front`** — Always pop from the front; the deque is sorted by distance.

14. Edge Cases

- **Source = destination** → distance = 0
- **Single node** → distance = 0
- **No path exists** → `dist[dest] = INF`
- **All edges are 0-weight** → 0-1 BFS works like BFS but pushes all to front; still $O(V+E)$
- **All edges are 1-weight** → 0-1 BFS works like standard BFS (all pushed to back)
- **Graph with 0-weight cycles** → 0-1 BFS handles them correctly; distance doesn't decrease after first visit
- **Disconnected graph** → unreachable nodes remain INF

15. Variations

Variation	What Changes	When Used	Importance
Dial's Algorithm	Uses bucket queue for integer weights up to <code>maxWeight</code>	Small integer weights (not just 0/1)	Medium — generalization of 0-1 BFS
BFS on 2-colors	Two queues instead of deque; alternate between them	Simpler implementation for 0-1 BFS	Low — pedagogical
Multi-source 0-1 BFS	Push all sources to deque initially	Multiple starting points	Medium — common in grid problems

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
BFS	Unweighted graph (all weights equal)	Use BFS when all edges have same weight
Dijkstra	Generalization for any non-negative weights	Use Dijkstra when weights are arbitrary non-negative integers
Dial's Algorithm	Bucket-based for small integer weights	Use Dial's when max weight is small (e.g., ≤ 100)
A*	Heuristic-based shortest path	Use A* when you have a good heuristic

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Minimum Cost to Make at Least One Valid Path in a Grid	LeetCode 1368	0-1 BFS on grid with direction signs	Easy
Shortest Path in Binary Matrix	LeetCode 1091	Can be solved with 0-1 BFS (all 1-cost moves)	Easy

Medium

Problem	Platform	Pattern	Difficulty
Minimum Number of Flips to Convert Binary Matrix to Zero Matrix	LeetCode 1284	0-1 BFS on state space	Medium
Shortest Path in a Grid with Obstacles Elimination	LeetCode 1293	BFS with state (k obstacles allowed) — can use 0-1 BFS variant	Medium

Hard

Problem	Platform	Pattern	Difficulty
Minimum Cost to Reach Destination with K Changes	GFG	0-1 BFS with state	Hard
Shortest Path with Power	CodeChef	0-1 BFS with power-ups	Hard

18. Interview Explanation

"0-1 BFS is an optimization for finding shortest paths in graphs where edge weights are only 0 or 1. Instead of a priority queue, we use a deque. When we process a node, if the edge weight is 0, we push the neighbor to the front of the deque — this means it gets processed immediately, at the same distance level. If the weight is 1, we push to the back. This maintains the invariant that the deque is always sorted by distance. The complexity is $O(V + E)$, which is better than Dijkstra's $O((V + E) \log V)$ for this special case."

19. Revision Notes

- **Key idea:** Deque with 0-weight → front, 1-weight → back
- **Time:** $O(V + E)$
- **Space:** $O(V)$
- **Data structure:** deque
- **No visited array:** Use distance comparison instead
- **No lazy deletion:** Deque ordering ensures correctness
- **Works for:** Weights $\in \{0, 1\}$ only
- **Generalization:** Dial's algorithm for small integer weights

20. Final Cheat Sheet

0-1 BFS

WHEN: Edge weights are only 0 or 1

COMPLEXITY: $O(V + E)$ time, $O(V)$ space

DATA STRUCTURE: Deque (double-ended queue)

TEMPLATE:

```
dist[src] = 0
dq.push_front(src)
while (!dq.empty()):
    u = dq.front(); dq.pop_front()
    for (v, w) in adj[u]:
        if (dist[u] + w < dist[v]):
            dist[v] = dist[u] + w
            if (w == 0): dq.push_front(v)
            else: dq.push_back(v)
```

KEY: 0-weight $\rightarrow$ front, 1-weight $\rightarrow$ back

COMPARE: Faster than Dijkstra for this special case

FAILS WITH: Weights other than 0 or 1

Shortest Path in DAG

1. Overview

Finding the shortest path in a **Directed Acyclic Graph (DAG)** is one of the simplest shortest path problems. Because a DAG has no cycles, we can use **topological sorting** to process nodes in a linear order, relaxing each node's outgoing edges exactly once. This gives $O(V + E)$ time — the fastest possible for shortest paths.

2. Intuition

Imagine you have a list of tasks where some tasks must be completed before others (dependencies). You want to find the minimum time to complete each task. Since there are no cycles, you can topologically sort the tasks (order them so that all dependencies come before the tasks that depend on them). Then, you simply go through the tasks in order, and for each task, you update the time for tasks that depend on it.

Why it works: In a DAG, a topological order ensures that when we process a node, all paths to it have already been considered (because any predecessor comes before it in the topological order). So we don't need multiple passes or priority queues — one pass through the topological order is sufficient.

3. When to Use It

- **Graph is a DAG** (directed and acyclic)
- **Need shortest (or longest) path in a DAG**
- **Task scheduling with dependencies** (PERT charts, critical path)
- **DP on DAG** — many DP problems can be modeled as shortest path in a DAG
- **Graph is directed and you know it has no cycles**

- **Need to find longest path in a DAG** (just negate weights or use -1 instead of weights)

Trigger phrases: "DAG", "topological sort", "acyclic", "dependency graph", "scheduling", "longest path in DAG", "critical path"

4. When Not to Use It

- **Graph has cycles** — topological sort is impossible; use Dijkstra or Bellman-Ford
- **Graph is undirected** — an undirected graph without cycles is a tree; use DFS/BFS
- **Graph is small and dense** — Floyd-Warshall might be simpler
- **Need all-pairs shortest paths** — Run topological sort once, then run DP from each source ($O(V(V+E))$) or use Floyd-Warshall

5. Core Concepts

5.1 Topological Sort

A linear ordering of vertices such that for every directed edge (u, v) , u comes before v . This is the foundation of the algorithm.

5.2 Kahn's Algorithm (BFS-based Topological Sort)

Use a queue to process nodes with in-degree 0. Remove them and their outgoing edges, updating in-degrees.

5.3 DFS-based Topological Sort

Run DFS, maintain a stack of finished nodes, and reverse the order. Alternative to Kahn's algorithm.

5.4 Relaxation in Topological Order

Once we have the topological order, we process nodes in that order. For each node, we relax all its outgoing edges. Since all predecessors come before the node, `dist[u]` is finalized when we process node u .

6. Step-by-Step Algorithm

```

Input: DAG  $G(V, E)$ , source  $s$ 
Output: Shortest distance from  $s$  to all nodes

1. Compute topological order of the graph.
   (Using Kahn's algorithm or DFS)

2. Initialize:
   -  $\text{dist}[v] = \text{INF}$  for all  $v$ ,  $\text{dist}[s] = 0$ 

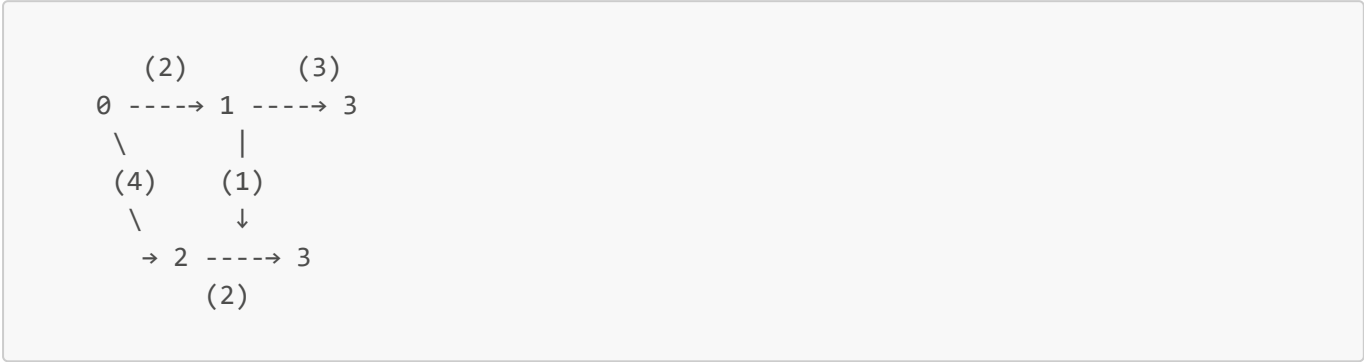
3. For each node  $u$  in topological order:
   - If  $\text{dist}[u] == \text{INF}$ : continue (unreachable from source)
   - For each neighbor  $v$  of  $u$  with weight  $w$ :
     - If  $\text{dist}[u] + w < \text{dist}[v]$ :
       -  $\text{dist}[v] = \text{dist}[u] + w$ 

```

4. `dist[]` contains shortest distances.
If `dist[v] == INF`, `v` is unreachable from source.

7. Dry Run

DAG:



Edges: $0 \rightarrow 1(2)$, $0 \rightarrow 2(4)$, $1 \rightarrow 2(1)$, $1 \rightarrow 3(3)$, $2 \rightarrow 3(2)$

Topological order: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$

Source = 0

Node	dist before	Outgoing edges	dist after
0	0	$0 \rightarrow 1(2)$: $\text{dist}[1] = \min(\text{INF}, 0+2) = 2$; $0 \rightarrow 2(4)$: $\text{dist}[2] = \min(\text{INF}, 0+4) = 4$	$\text{dist}[0] = 0$, $\text{dist}[1] = 2$, $\text{dist}[2] = 4$
1	2	$1 \rightarrow 2(1)$: $\text{dist}[2] = \min(4, 2+1) = 3$; $1 \rightarrow 3(3)$: $\text{dist}[3] = \min(\text{INF}, 2+3) = 5$	$\text{dist}[1] = 2$, $\text{dist}[2] = 3$, $\text{dist}[3] = 5$
2	3	$2 \rightarrow 3(2)$: $\text{dist}[3] = \min(5, 3+2) = 5$	$\text{dist}[2] = 3$, $\text{dist}[3] = 5$
3	5	—	$\text{dist}[3] = 5$

Final distances: `[0, 2, 3, 5]`

Path to 3: $0 \rightarrow 1 \rightarrow 3$ (cost 5) or $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ (cost 5)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using pii = pair<int, int>;

const ll INF = 1e18;

// Topological sort using Kahn's algorithm (BFS)
vector<int> topological_sort(const vector<vector<int>>& adj, const vector<int>&
```

```

indeg) {
    int n = adj.size();
    vector<int> indeg_copy = indeg;
    queue<int> q;
    vector<int> topo;

    for (int i = 0; i < n; i++) {
        if (indeg_copy[i] == 0) q.push(i);
    }

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        topo.push_back(u);

        for (int v : adj[u]) {
            if (--indeg_copy[v] == 0) q.push(v);
        }
    }

    // If topo.size() != n, graph has a cycle
    return topo;
}

// Shortest path in DAG
// Returns vector of distances from source

vector<ll> shortest_path_dag(const vector<vector<pii>>& adj, int src) {
    int n = adj.size();

    // Build adjacency list for topological sort (without weights)
    vector<vector<int>> adj_topo(n);
    vector<int> indeg(n, 0);
    for (int u = 0; u < n; u++) {
        for (auto [v, w] : adj[u]) {
            adj_topo[u].push_back(v);
            indeg[v]++;
        }
    }

    // Get topological order
    vector<int> topo = topological_sort(adj_topo, indeg);
    if ((int)topo.size() != n) {
        // Graph has a cycle – not a DAG
        return {};
    }

    // Initialize distances
    vector<ll> dist(n, INF);
    dist[src] = 0;

    // Process nodes in topological order
    for (int u : topo) {
        if (dist[u] == INF) continue; // Unreachable from source
    }
}

```

```

        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }

    return dist;
}

// Example usage
int main() {
    int n = 4;
    vector<vector<pii>> adj(n);

    adj[0] = {{1, 2}, {2, 4}};
    adj[1] = {{2, 1}, {3, 3}};
    adj[2] = {{3, 2}};
    adj[3] = {};

    vector<ll> dist = shortest_path_dag(adj, 0);

    if (dist.empty()) {
        cout << "Graph has a cycle!\n";
        return 0;
    }

    cout << "Shortest distances from 0:\n";
    for (int i = 0; i < n; i++) {
        if (dist[i] == INF) cout << "  To " << i << ": INF\n";
        else cout << "  To " << i << ": " << dist[i] << "\n";
    }

    return 0;
}

```

9. Python Implementation

```

from collections import deque
from typing import List, Tuple

INF = 10**18

def topological_sort(adj: List[List[int]], indeg: List[int]) -> List[int]:
    n = len(adj)
    indeg = indeg.copy()
    q = deque([i for i in range(n) if indeg[i] == 0])
    topo = []

    while q:

```

```

        u = q.popleft()
        topo.append(u)
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)

    return topo

def shortest_path_dag(adj: List[List[Tuple[int, int]]], src: int):
    n = len(adj)

    # Build adjacency for topological sort
    adj_topo = [[] for _ in range(n)]
    indeg = [0] * n
    for u in range(n):
        for v, w in adj[u]:
            adj_topo[u].append(v)
            indeg[v] += 1

    topo = topological_sort(adj_topo, indeg)
    if len(topo) != n:
        raise ValueError("Graph has a cycle")

    dist = [INF] * n
    dist[src] = 0

    for u in topo:
        if dist[u] == INF:
            continue
        for v, w in adj[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w

    return dist

# Example usage
if __name__ == "__main__":
    n = 4
    adj = [[] for _ in range(n)]
    adj[0] = [(1, 2), (2, 4)]
    adj[1] = [(2, 1), (3, 3)]
    adj[2] = [(3, 2)]
    adj[3] = []

    try:
        dist = shortest_path_dag(adj, 0)
        print("Shortest distances from 0:")
        for i in range(n):
            if dist[i] == INF:
                print(f"    To {i}: INF")
            else:

```

```

        print(f" To {i}: {dist[i]}")
    except ValueError as e:
        print(f"Error: {e}")

```

10. Code Explanation

- **Topological sort:** We use Kahn's algorithm (BFS-based). We track in-degrees, start with nodes that have in-degree 0, process them, and reduce in-degrees of their neighbors.
- **Cycle detection:** If `topo.size() != n`, the graph has a cycle. A DAG must have a valid topological ordering of all nodes.
- **Distance initialization:** `dist[src] = 0`, all others `INF`.
- **Processing in topological order:** For each node `u` in topological order, if it's reachable (`dist[u] != INF`), we relax all its outgoing edges. Since all predecessors of `u` come before `u` in the topological order, `dist[u]` is already finalized.
- **Unreachable nodes:** If `dist[u] == INF`, we skip it — no path from source to `u` exists.

11. Complexity Analysis

Operation	Complexity
Topological Sort (Kahn's)	$O(V + E)$
Shortest Path DP	$O(V + E)$
Total Time	$O(V + E)$
Space	$O(V)$ — for dist, topo, indeg

- **Best case:** $O(V + E)$ — always
- **Worst case:** $O(V + E)$ — no extra overhead
- **This is optimal:** No algorithm can find shortest paths in a DAG faster than $O(V + E)$ because you need to examine all edges at least once.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Longest path in DAG	"Longest path", "critical path", "maximum time"	Negate weights (or use <code>max</code> instead of <code>min</code>)	Critical Path in PERT, Longest Path in DAG
DP on DAG	Problem can be modeled as graph with dependencies	Topological sort + DP (relaxation)	Longest Increasing Path in Matrix, Minimum Time to Complete All Tasks
Number of paths in DAG	"Count number of paths from source to destination"	Similar DP but count ways instead of distances	Count paths in DAG

Pattern	How to Identify	Approach	Example Problems
Minimum time to complete all tasks	Tasks with dependencies, each takes time	DAG where each node weight = task time; edge weight = 0	Parallel task scheduling
Maximum profit with deadlines	Jobs with deadlines, dependencies	Model as DAG, find longest path	Job scheduling with precedence

13. Common Mistakes

- **Not checking for cycles** — If the graph has a cycle, topological sort will not include all nodes, and the algorithm will produce wrong results.
- **Processing nodes not in topological order** — If you process nodes in arbitrary order, you might process a node before its predecessors, leading to incorrect distances.
- **Not handling unreachable source** — If the source is not the first node in topological order, nodes before it are unreachable (dist = INF). Skip them correctly.
- **Using the algorithm for undirected graphs** — An undirected graph without cycles is a tree, not a DAG. Use DFS/BFS for trees.
- **Forgetting that topological order may not start with source** — The source may not be the first node in topological order. Nodes before the source are unreachable.
- **Using `dist[u] == INF` check** — Always check before relaxing; otherwise, `INF + w` can overflow.
- **Not handling disconnected DAGs** — A DAG may have multiple components; topological sort still works.

14. Edge Cases

- **Single node** → topological order = [0], distance = 0
- **Empty graph (no edges)** → topological order = [0, 1, ..., V-1], distances from source
- **Source is a sink (no outgoing edges)** → distance to source = 0, all others INF
- **Source is not reachable to some nodes** → those nodes remain INF
- **Graph with multiple components** → topological sort works, distances from source only in its component
- **Graph with parallel edges** → all edges are relaxed; the shortest weight wins
- **Source is unreachable from any node before it in topological order** → skip those nodes

15. Variations

Variation	What Changes	When Used	Importance
Longest path in DAG	Use <code>max</code> instead of <code>min</code> , initialize dist to -INF	Find longest path (critical path in project management)	High — important for scheduling problems
Count paths in DAG	<code>ways[v] += ways[u]</code> instead of relaxing	Count number of paths from source to each node	Medium — common in DP problems
Minimum path with exactly k	Can use DP with matrix exponentiation	Need paths with exactly k edges	Low — niche

Variation	What Changes	When Used	Importance
edges			
DFS-based topological sort	Use DFS post-order traversal	Alternative to Kahn's algorithm	Medium — both are fine

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Dijkstra	General shortest path in graphs with cycles	Use when graph has cycles (not a DAG)
Bellman-Ford	Handles negative weights and cycles	Use when graph has cycles and negative weights
DFS	Can also find topological sort (post-order)	Use DFS-based topological sort for simpler code
DP with memoization	Many DP problems are shortest paths in implicit DAGs	Use when DAG is implicit (e.g., grid DP)

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Shortest Path in DAG	GFG	Standard shortest path in DAG	Easy
Longest Path in DAG	GFG	Longest path in DAG (negate weights)	Easy

Medium

Problem	Platform	Pattern	Difficulty
Minimum Time to Complete All Tasks	GFG	DAG shortest path with task dependencies	Medium
Course Schedule II	LeetCode 210	Topological sort + DP for shortest completion time	Medium

Hard

Problem	Platform	Pattern	Difficulty
Longest Increasing Path in a Matrix	LeetCode 329	Implicit DAG (grid), DFS + memoization	Hard
Parallel Courses III	LeetCode 2050	DAG shortest path (longest path for minimum time)	Hard

18. Interview Explanation

"Shortest path in a DAG is the simplest shortest path problem. Since there are no cycles, we can topologically sort the graph and then process nodes in that order. For each node, we relax all its outgoing edges. Because of the topological ordering, when we process a node, all its incoming paths have already been considered, so we don't need multiple passes. The time complexity is $O(V + E)$, which is optimal. To find the longest path in a DAG, we can either negate all weights or replace `min` with `max`."

19. Revision Notes

- **Key idea:** Topological sort + one pass of relaxation
- **Time:** $O(V + E)$ — optimal
- **Space:** $O(V)$
- **Two steps:** (1) Topological sort, (2) Process in topological order, relax edges
- **Cycle detection:** If topological sort doesn't include all nodes, there's a cycle
- **Longest path:** Use `max` instead of `min`, initialize to `-INF`
- **No priority queue needed:** The topological order provides the correct processing sequence
- **Unreachable nodes:** Nodes before source in topological order are unreachable

20. Final Cheat Sheet

SHORTEST PATH IN DAG

WHEN: Graph is a Directed Acyclic Graph

COMPLEXITY: $O(V + E)$ time, $O(V)$ space

DATA STRUCTURE: Topological order (queue or stack)

TEMPLATE:

```
// Step 1: Topological sort
topo = topological_sort(adj)

// Step 2: Process in topological order
dist[src] = 0
for (u in topo):
    if (dist[u] == INF) continue
    for (v, w) in adj[u]:
        if (dist[u] + w < dist[v]):
            dist[v] = dist[u] + w
```

KEY: Topological order ensures all predecessors processed first

LONGEST PATH: Use `max()` instead of `min()`, initialize to `-INF`

CYCLE CHECK: `topo.size() != n` → cycle exists

Johnson's Algorithm

1. Overview

Johnson's algorithm finds the **all-pairs shortest path** in a sparse graph, even with **negative edge weights** (but no negative cycles). It combines **Bellman-Ford** (to handle negative weights and compute potentials) with **Dijkstra** (to find shortest paths from each node). For sparse graphs, it is faster than Floyd-Warshall.

2. Intuition

The problem: Dijkstra is fast ($O((V+E) \log V)$ per source) but doesn't work with negative weights. Bellman-Ford handles negative weights but is slow ($O(V \cdot E)$ per source). Can we somehow transform the graph so that all edge weights are non-negative, then run Dijkstra from each node?

Johnson's insight: We can **reweight** the graph using a **potential function** $h(v)$. For each edge (u, v, w) , we compute a new weight $w' = w + h(u) - h(v)$. The key property: shortest paths in the reweighted graph are the same as in the original graph! The difference is that the new weights are non-negative, so we can use Dijkstra.

How to find $h(v)$? Add a new "super source" node connected to all nodes with 0-weight edges. Run Bellman-Ford from this super source. The distances $h(v)$ from the super source are the potentials. Due to the triangle inequality, $h(v) \leq h(u) + w(u, v)$, so $w' = w + h(u) - h(v) \geq 0$.

3. When to Use It

- **All-pairs shortest path** in a **sparse graph** with **negative edge weights**
- **Graph is too large for Floyd-Warshall** ($V > 500$ but $E \approx V$)
- **Graph has negative edges but no negative cycles**
- **Need the actual shortest paths** (not just distances)
- **When you need to run Dijkstra from multiple sources** on a graph with negative weights

Trigger phrases: "all-pairs shortest path with negative weights", "sparse graph with negative edges", "Johnson's algorithm"

4. When Not to Use It

- **Graph is dense** — Floyd-Warshall's $O(V^3)$ is simpler and may be faster
- **Graph has no negative edges** — Running Dijkstra V times is simpler (skip Bellman-Ford step)
- **Graph is unweighted** — Running BFS V times is $O(V(V+E))$
- **Graph has negative cycles** — Johnson's algorithm cannot handle them (Bellman-Ford will detect them)
- **Only need single-source shortest path** — Use Bellman-Ford directly
- **V is huge** — $O(V^2 \log V + V \cdot E)$ is still too much for large V

5. Core Concepts

5.1 Potential Function

A function $h(v)$ assigning a real number to each vertex. Used to reweight edges.

5.2 Reweighting

New edge weight: $w'(u, v) = w(u, v) + h(u) - h(v)$. This transformation preserves shortest paths: the total weight of any path from s to t changes by $h(s) - h(t)$, which is constant for all paths between the same pair.

5.3 Super Source

A new node connected to all original nodes with 0-weight edges. Running Bellman-Ford from this super source gives the potentials $h(v)$ = shortest distance from the super source to v .

5.4 Non-Negative Guarantee

By the triangle inequality of shortest paths: $h(v) \leq h(u) + w(u, v)$, so $w'(u, v) = w(u, v) + h(u) - h(v) \geq 0$. All reweighted edges are non-negative.

6. Step-by-Step Algorithm

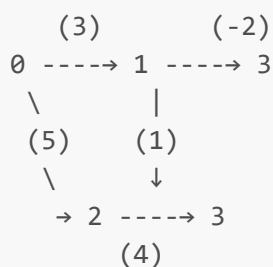
Input: Graph $G(V, E)$ with edge weights (may be negative)

Output: All-pairs shortest distance matrix

1. Add a new node s (super source) connected to all original nodes with 0-weight edges.
2. Run Bellman-Ford from s :
 - If negative cycle detected $\rightarrow$ stop (no solution)
 - Let $h(v)$ = shortest distance from s to v
3. Reweight all original edges:
 - $w'(u, v) = w(u, v) + h(u) - h(v)$
4. For each node u in the original graph:
 - Run Dijkstra from u on the reweighted graph
 - Let $d'(u, v)$ = shortest distance from u to v in reweighted graph
5. Compute original distances:
 - $d(u, v) = d'(u, v) - h(u) + h(v)$
6. If $d(u, v) == \text{INF}$, v is unreachable from u .

7. Dry Run

Graph:



Edges: $0 \rightarrow 1(3)$, $0 \rightarrow 2(5)$, $1 \rightarrow 2(1)$, $1 \rightarrow 3(-2)$, $2 \rightarrow 3(4)$

Step 1: Add super source (node 4) with edges to all nodes with weight 0.

Step 2: Bellman-Ford from super source 4.

Node	$h(v)$ = distance from 4
0	0
1	0
2	0
3	0

(Because all edges from super source have weight 0, and there are no negative edges from super source, all distances are 0.)

Wait — let me recalculate. The super source connects to all nodes with weight 0. So $\text{dist}[4] = 0$, $\text{dist}[0] = 0$, $\text{dist}[1] = 0$, $\text{dist}[2] = 0$, $\text{dist}[3] = 0$ from the direct edges. But what about paths through other nodes? From $4 \rightarrow 0 \rightarrow 1$: $\text{dist}[1] = 0 + 3 = 3$, which is > 0 , so no update. So all $h(v) = 0$.

Step 3: Reweight edges. Since all $h(v) = 0$, $w'(u,v) = w(u,v) + 0 - 0 = w(u,v)$. No change in this case.

Step 4: Dijkstra from each node. From 0: $\text{dist}[0]=0$, $\text{dist}[1]=3$, $\text{dist}[2]=4$ (via 1), $\text{dist}[3]=1$ (via 1) From 1: $\text{dist}[1]=0$, $\text{dist}[2]=1$, $\text{dist}[3]=-2$ From 2: $\text{dist}[2]=0$, $\text{dist}[3]=4$ From 3: $\text{dist}[3]=0$

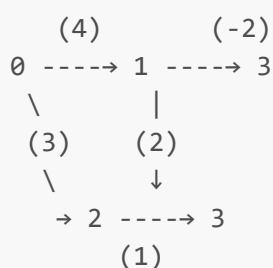
Step 5: Compute original distances. Since $h(v) = 0$, $d(u,v) = d'(u,v)$.

Final distance matrix:

i \ j	0	1	2	3
0	0	3	4	1
1	INF	0	1	-2
2	INF	INF	0	4
3	INF	INF	INF	0

Let me use a better example with negative edges that actually need reweighting.

Better example:



Edges: $0 \rightarrow 1(4)$, $0 \rightarrow 2(3)$, $1 \rightarrow 2(2)$, $1 \rightarrow 3(-2)$, $2 \rightarrow 3(1)$

Step 1: Super source S = 4. Edges: $4 \rightarrow 0(0)$, $4 \rightarrow 1(0)$, $4 \rightarrow 2(0)$, $4 \rightarrow 3(0)$.

Step 2: Bellman-Ford from 4. $\text{dist}[4] = 0$ $\text{dist}[0] = 0$ (via $4 \rightarrow 0$) $\text{dist}[1] = 0$ (via $4 \rightarrow 1$) $\text{dist}[2] = 0$ (via $4 \rightarrow 2$) $\text{dist}[3] = 0$ (via $4 \rightarrow 3$) But wait, can we reach 3 with a shorter path? $4 \rightarrow 0 \rightarrow 1 \rightarrow 3$: $0 + 4 - 2 = 2$. That's > 0 , so no. $4 \rightarrow 1 \rightarrow 3$: $0 - 2 = -2$. That's < 0 , so $\text{dist}[3] = -2$! $4 \rightarrow 0 \rightarrow 2 \rightarrow 3$: $0 + 3 + 1 = 4$. No. $4 \rightarrow 1 \rightarrow 2 \rightarrow 3$: $0 + 2 + 1 = 3$. No.

Iteration	Updates
Init	$h = [0, 0, 0, 0, 0]$
1	$4 \rightarrow 1(0)$: no change; $4 \rightarrow 2(0)$: no change; $4 \rightarrow 3(0)$: no change; $1 \rightarrow 3(-2)$: $h[3] = \min(0, 0-2) = -2$
2	$1 \rightarrow 3(-2)$: $h[3]$ stays -2 ; no more changes

So $h = [0, 0, 0, -2, 0]$ for nodes $[0, 1, 2, 3, 4]$. We only need h for original nodes.

Step 3: Reweight edges. $w'(u,v) = w(u,v) + h(u) - h(v)$

- $0 \rightarrow 1(4)$: $w' = 4 + 0 - 0 = 4$
- $0 \rightarrow 2(3)$: $w' = 3 + 0 - 0 = 3$
- $1 \rightarrow 2(2)$: $w' = 2 + 0 - 0 = 2$
- $1 \rightarrow 3(-2)$: $w' = -2 + 0 - (-2) = 0$ ✓ (non-negative)
- $2 \rightarrow 3(1)$: $w' = 1 + 0 - (-2) = 3$

All reweighted edges are non-negative!

Step 4: Dijkstra from each node on reweighted graph. From 0: $d' = [0, 4, 3, 3]$ ($0 \rightarrow 2 \rightarrow 3$: $3+3=6$, but $0 \rightarrow 1 \rightarrow 3$: $4+0=4$... wait)

Let me recalculate. Reweighted graph: $0 \rightarrow 1(4)$, $0 \rightarrow 2(3)$, $1 \rightarrow 2(2)$, $1 \rightarrow 3(0)$, $2 \rightarrow 3(3)$

Dijkstra from 0:

- $\text{dist}[0]=0$, $\text{dist}[1]=4$, $\text{dist}[2]=3$
- Process 2: $\text{dist}[3] = \min(\text{INF}, 3+3=6)$
- Process 1: $\text{dist}[3] = \min(6, 4+0=4) \rightarrow \text{dist}[3]=4$
- $d'(0,0)=0$, $d'(0,1)=4$, $d'(0,2)=3$, $d'(0,3)=4$

Dijkstra from 1:

- $\text{dist}[1]=0$, $\text{dist}[2]=2$, $\text{dist}[3]=0$
- $d'(1,1)=0$, $d'(1,2)=2$, $d'(1,3)=0$

Dijkstra from 2:

- $\text{dist}[2]=0$, $\text{dist}[3]=3$
- $d'(2,2)=0$, $d'(2,3)=3$

Dijkstra from 3:

- $\text{dist}[3]=0$
- $d'(3,3)=0$

Step 5: Convert back to original distances. $d(u,v) = d'(u,v) - h(u) + h(v)$

- $d(0,1) = 4 - 0 + 0 = 4$
- $d(0,2) = 3 - 0 + 0 = 3$
- $d(0,3) = 4 - 0 + (-2) = 2$ ✓ (path $0 \rightarrow 1 \rightarrow 3$: $4 - 2 = 2$)
- $d(1,2) = 2 - 0 + 0 = 2$
- $d(1,3) = 0 - 0 + (-2) = -2$ ✓
- $d(2,3) = 3 - 0 + (-2) = 1$ ✓

Final distance matrix:

i\j	0	1	2	3
0	0	4	3	2
1	INF	0	2	-2
2	INF	INF	0	1
3	INF	INF	INF	0

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using pii = pair<int, int>;

const ll INF = 1e18;

// Johnson's Algorithm: All-Pairs Shortest Path
// Handles negative weights (no negative cycles)
// Faster than Floyd-Warshall for sparse graphs

struct Edge {
    int u, v;
    ll w;
};

vector<vector<ll>> johnson(int n, vector<Edge>& edges) {
    // Step 1: Add super source (node n)
    vector<Edge> ext_edges = edges;
    for (int i = 0; i < n; i++) {
        ext_edges.push_back({n, i, 0});
    }

    // Step 2: Bellman-Ford from super source to get potentials h
    int N = n + 1;
    vector<ll> h(N, INF);
    h[n] = 0;

    for (int i = 0; i < N - 1; i++) {
        bool updated = false;
        for (auto [u, v, w] : ext_edges) {
```

```

        if (h[u] != INF && h[u] + w < h[v]) {
            h[v] = h[u] + w;
            updated = true;
        }
    }
    if (!updated) break;
}

// Check for negative cycles
for (auto [u, v, w] : ext_edges) {
    if (h[u] != INF && h[u] + w < h[v]) {
        // Negative cycle detected
        return {};
    }
}

// Step 3: Reweight edges
vector<vector<pii>> adj(n);
for (auto [u, v, w] : edges) {
    ll new_w = w + h[u] - h[v];
    adj[u].push_back({v, new_w}); // new_w >= 0
}

// Step 4: Run Dijkstra from each node on reweighted graph
// Step 5: Convert back to original distances
vector<vector<ll>> dist(n, vector<ll>(n, INF));

for (int src = 0; src < n; src++) {
    // Dijkstra
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    dist[src][src] = 0;
    pq.push({0, src});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d != dist[src][u]) continue;

        for (auto [v, w] : adj[u]) {
            if (dist[src][u] + w < dist[src][v]) {
                dist[src][v] = dist[src][u] + w;
                pq.push({dist[src][v], v});
            }
        }
    }
}

// Convert back to original weight space
for (int v = 0; v < n; v++) {
    if (dist[src][v] != INF) {
        dist[src][v] = dist[src][v] - h[src] + h[v];
    }
}
}

```

```

    return dist;
}

// Example usage
int main() {
    int n = 4;
    vector<Edge> edges = {
        {0, 1, 4},
        {0, 2, 3},
        {1, 2, 2},
        {1, 3, -2},
        {2, 3, 1}
    };

    vector<vector<ll>> dist = johnson(n, edges);

    if (dist.empty()) {
        cout << "Negative cycle detected!\n";
        return 0;
    }

    cout << "All-Pairs Shortest Distances:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][j] == INF) cout << "INF\t";
            else cout << dist[i][j] << "\t";
        }
        cout << "\n";
    }

    return 0;
}

```

9. Python Implementation

```

import heapq
from typing import List, Tuple

INF = 10**18

def johnson(n: int, edges: List[Tuple[int, int, int]]):
    # Step 1: Add super source
    ext_edges = edges + [(n, i, 0) for i in range(n)]
    N = n + 1

    # Step 2: Bellman-Ford from super source
    h = [INF] * N
    h[n] = 0

    for _ in range(N - 1):

```

```

        updated = False
        for u, v, w in ext_edges:
            if h[u] != INF and h[u] + w < h[v]:
                h[v] = h[u] + w
                updated = True
        if not updated:
            break

# Check for negative cycles
for u, v, w in ext_edges:
    if h[u] != INF and h[u] + w < h[v]:
        raise ValueError("Negative cycle detected")

# Step 3: Reweight edges
adj = [[] for _ in range(n)]
for u, v, w in edges:
    new_w = w + h[u] - h[v]
    adj[u].append((v, new_w))

# Step 4 & 5: Run Dijkstra from each node, convert back
dist = [[INF] * n for _ in range(n)]

for src in range(n):
    dist[src][src] = 0
    pq = [(0, src)]

    while pq:
        d, u = heapq.heappop(pq)
        if d != dist[src][u]:
            continue
        for v, w in adj[u]:
            if dist[src][u] + w < dist[src][v]:
                dist[src][v] = dist[src][u] + w
                heapq.heappush(pq, (dist[src][v], v))

# Convert back
for v in range(n):
    if dist[src][v] != INF:
        dist[src][v] = dist[src][v] - h[src] + h[v]

return dist

# Example usage
if __name__ == "__main__":
    n = 4
    edges = [
        (0, 1, 4),
        (0, 2, 3),
        (1, 2, 2),
        (1, 3, -2),
        (2, 3, 1)
    ]

```

```

try:
    dist = johnson(n, edges)
    print("All-Pairs Shortest Distances:")
    for i in range(n):
        row = "\t".join(str(d) if d != INF else "INF" for d in dist[i])
        print(row)
except ValueError as e:
    print(f"Error: {e}")

```

10. Code Explanation

- **Super source (node n):** A new node connected to all original nodes with weight 0. This ensures that $h(v)$ is the shortest distance from the super source to v , which satisfies the triangle inequality.
- **Bellman-Ford:** Runs on the extended graph (with super source). If a negative cycle is detected, the algorithm stops.
- **Potentials $h(v)$:** The shortest distances from the super source. These are used to reweight edges.
- **Reweighting:** $w' = w + h(u) - h(v)$. The new weight is guaranteed to be non-negative.
- **Dijkstra V times:** For each source, run Dijkstra on the reweighted graph. Store distances in a 2D matrix.
- **Convert back:** $d(u, v) = d'(u, v) - h(u) + h(v)$. This reverses the effect of reweighting.

11. Complexity Analysis

Operation	Complexity
Bellman-Ford	$O(V \cdot E)$ — one pass over all edges V times
Reweighting	$O(E)$ — one pass over all edges
$V \times$ Dijkstra	$O(V(E + V \log V))$ — using binary heap
Total Time	$O(V \cdot E + V(E + V \log V)) = O(V \cdot E + V^2 \log V)$
Space	$O(V^2)$ — for the distance matrix

- **Sparse graph ($E \approx V$):** $O(V^2 \log V)$ — better than Floyd-Warshall's $O(V^3)$
- **Dense graph ($E \approx V^2$):** $O(V^3)$ — same as Floyd-Warshall but with more overhead
- **Best for:** Sparse graphs with $V \leq 1000$ and $E \approx V$

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
All-pairs with negative weights	Need all-pairs distances, graph has negative edges	Johnson's algorithm	Various CP problems
Sparse graph all-pairs	V is large (up to 1000), $E \approx V$	Johnson's algorithm	Road networks, communication networks
Reweighting technique	Need to use Dijkstra but graph has negative edges	Bellman-Ford for potentials, then Dijkstra	Any graph with negative edges

13. Common Mistakes

- **Not detecting negative cycles** — Always check after Bellman-Ford. If a negative cycle exists, Johnson's algorithm doesn't work.
- **Forgetting to convert back** — The distances from Dijkstra are in the reweighted space. You must convert back: $d(u,v) = d'(u,v) - h(u) + h(v)$.
- **Using wrong super source connections** — The super source must connect to all original nodes with weight 0.
- **Incorrect INF handling** — When converting back, if `dist[src][v] == INF`, skip the conversion (it stays INF).
- **Overflow in reweighting** — $w + h(u) - h(v)$ can overflow if h values are large. Use `long long`.
- **Not running Bellman-Ford on extended graph** — The super source must be included in the Bellman-Ford run.
- **Forgetting that the super source is temporary** — Don't include the super source in the final distance matrix.

14. Edge Cases

- **Single node** → distance matrix = `[[0]]`
- **No negative edges** → Johnson's works but is overkill; just run Dijkstra V times
- **Graph with negative cycle** → detect and stop
- **Disconnected graph** → INF for unreachable pairs
- **Graph with zero-weight edges** → Johnson's handles them
- **Large h values** → $h(v)$ can be negative, but reweighting still works

15. Variations

Variation	What Changes	When Used	Importance
Johnson's with Fibonacci Heap	Dijkstra uses Fibonacci heap	Theoretical $O(V^2 \log V + V \cdot E)$	Low — Fibonacci heap is impractical
Johnson's without super source	If no negative edges, skip Bellman-Ford, use $h=0$	Graph has no negative edges	Medium — just run Dijkstra V times
Johnson's for longest paths	Negate weights, run Johnson's, then negate back	Need longest paths in graph with negative edges	Low — niche

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Floyd-Warshall	$O(V^3)$ all-pairs, handles negative weights	Choose for dense graphs ($E \approx V^2$)
Dijkstra (V times)	$O(V(E + V \log V))$ all-pairs, no negative weights	Choose when no negative edges
Bellman-Ford	Single-source with negative weights	Choose when only single-source needed

Algorithm	Connection	When to Choose
SPFA	Queue-based Bellman-Ford	Choose for faster single-source with negatives

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Johnson's Algorithm Implementation	GFG	Standard implementation	Easy
Shortest Path in Weighted Graph with Negative Edges	GFG	Single-source negative weights (Bellman-Ford)	Easy

Medium

Problem	Platform	Pattern	Difficulty
All-Pairs Shortest Path with Negative Weights	CodeChef	Johnson's algorithm	Medium
Shortest Path in a Graph with Negative Weights (Multiple Queries)	GFG	Johnson's algorithm for multiple queries	Medium

Hard

Problem	Platform	Pattern	Difficulty
Minimum Cost to Connect All Nodes with Negative Edges	GFG	Johnson's with negative edges	Hard
Road Network with Toll and Discount	Codeforces	All-pairs shortest path with reweighting	Hard

18. Interview Explanation

"Johnson's algorithm finds all-pairs shortest paths in a sparse graph with negative edge weights. The key insight is to reweight the graph so that all edges become non-negative, allowing us to use Dijkstra from each node. We first add a super source connected to all nodes with 0-weight edges and run Bellman-Ford to get potentials $h(v)$. Then we reweight each edge: $w' = w + h(u) - h(v)$, which is guaranteed to be non-negative. Finally, we run Dijkstra from each node on the reweighted graph and convert back: $d(u,v) = d'(u,v) - h(u) + h(v)$. The complexity is $O(V \cdot E + V^2 \log V)$, which is better than Floyd-Warshall's $O(V^3)$ for sparse graphs."

19. Revision Notes

- **Key idea:** Reweight graph to make all edges non-negative, then run Dijkstra V times
- **Three steps:** (1) Bellman-Ford for potentials, (2) Reweight edges, (3) Dijkstra from each node + convert back
- **Super source:** Node n connected to all nodes with weight 0

- **$h(v)$** : Shortest distance from super source to v (from Bellman-Ford)
- **Reweighting**: $w' = w + h(u) - h(v)$ (always ≥ 0)
- **Convert back**: $d(u, v) = d'(u, v) - h(u) + h(v)$
- **Time**: $O(V \cdot E + V^2 \log V)$
- **Space**: $O(V^2)$
- **Better than Floyd-Warshall for**: Sparse graphs ($E \approx V$)
- **Fails with**: Negative cycles

20. Final Cheat Sheet

JOHNSON'S ALGORITHM

WHEN: All-pairs shortest path, sparse graph, negative weights

COMPLEXITY: $O(V \cdot E + V^2 \log V)$ time, $O(V^2)$ space

HANDLES: Negative weights (no negative cycles)

TEMPLATE:

```
// 1. Add super source, run Bellman-Ford → h[v]
// 2. Reweight: w' = w + h[u] - h[v]
// 3. Dijkstra from each src:
//   d' = Dijkstra on reweighted graph
//   d[src][v] = d'[v] - h[src] + h[v]
```

KEY: Reweighting makes all edges non-negative

CONVERT BACK: $d(u, v) = d'(u, v) - h(u) + h(v)$

FAILS WITH: Negative cycles

CHOOSE: Over Floyd-Warshall for sparse graphs

A* Search Algorithm

1. Overview

A* (pronounced "A-star") is an **informed search** algorithm that finds the shortest path between a **single source and a single destination**. It uses a **heuristic** to guide the search toward the target, making it much faster than Dijkstra for large graphs when a good heuristic is available. It is widely used in GPS navigation, video games, and robotics.

2. Intuition

Imagine you are in a city and want to reach a specific destination. Dijkstra's algorithm would explore in all directions equally — like drawing concentric circles from your starting point. But you know the destination is to the northeast, so it makes sense to explore northeast first.

A* does exactly this. It uses a **heuristic function** $h(n)$ that estimates the distance from node n to the destination. At each step, it picks the node with the smallest $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost from the start to n , and $h(n)$ is the estimated cost from n to the goal.

Why it works: If the heuristic is **admissible** (never overestimates the true cost), A* is guaranteed to find the shortest path. The heuristic prunes the search space by focusing on promising directions.

3. When to Use It

- **Single pair shortest path** (not all-pairs or all-nodes)
- **Large graph** where Dijkstra is too slow
- **You have a good heuristic** (e.g., Euclidean distance, Manhattan distance)
- **Pathfinding in games** (grid-based movement)
- **GPS navigation** (road networks)
- **Robot motion planning** (configuration space)
- **15-puzzle / sliding puzzle** (heuristic: number of misplaced tiles)

Trigger phrases: "shortest path from s to t", "pathfinding", "heuristic", "informed search", "A-star", "GPS", "game AI pathfinding"

4. When Not to Use It

- **Need distances from source to all nodes** — Dijkstra is better (A* is optimized for a single target)
- **No good heuristic available** — Without a good heuristic, A* degenerates to Dijkstra ($h(n) = 0$)
- **Heuristic is expensive to compute** — If computing $h(n)$ costs more than the search itself, A* is worse
- **Heuristic is not admissible** — A* may not find the shortest path
- **Graph is small** — Dijkstra is simpler and sufficient
- **Graph has negative weights** — A* generally assumes non-negative weights
- **Memory is tight** — A* can use more memory than Dijkstra (it stores the open set)
- **All-pairs shortest path** — Use Floyd-Warshall or Johnson's

5. Core Concepts

5.1 $g(n)$ — Actual Cost from Start

This is the cost of the cheapest known path from the start node to node n . This is the same as Dijkstra's distance.

5.2 $h(n)$ — Heuristic Estimate

An estimate of the cost from node n to the goal. The heuristic must be **admissible** (never overestimates) for A* to guarantee optimality. Common heuristics:

- **Manhattan distance:** $|x_1 - x_2| + |y_1 - y_2|$ (grid with 4-directional movement)
- **Euclidean distance:** $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$ (any direction movement)
- **Chebyshev distance:** $\max(|x_1 - x_2|, |y_1 - y_2|)$ (8-directional movement)
- **Octile distance:** For grid with diagonal movement at cost $\sqrt{2}$
- **Zero heuristic:** $h(n) = 0$ (degenerates to Dijkstra)

5.3 $f(n) = g(n) + h(n)$ — Total Estimated Cost

The priority value used in the open set. A* always expands the node with the smallest $f(n)$.

5.4 Open Set (Priority Queue)

Nodes that have been discovered but not yet processed. The priority queue is ordered by $f(n)$.

5.5 Closed Set

Nodes that have been processed (expanded). In A^* with a consistent heuristic, once a node is expanded, its f -value is final.

5.6 Admissible Heuristic

A heuristic $h(n)$ is admissible if it never overestimates the actual cost to reach the goal: $h(n) \leq h^*(n)$ where $h^*(n)$ is the true shortest distance to the goal.

5.7 Consistent (Monotonic) Heuristic

A heuristic is consistent if $h(n) \leq c(n, m) + h(m)$ for all neighbors m of n . A consistent heuristic is always admissible. All consistent heuristics guarantee that A^* never needs to re-open a node.

6. Step-by-Step Algorithm

Input: Graph $G(V, E)$, source s , destination t , heuristic function $h(n)$
 Output: Shortest path from s to t , and its cost

1. Initialize:
 - $g[s] = 0$
 - $f[s] = h(s)$
 - open set (min-heap): push $(f[s], s)$
 - $\text{parent}[s] = -1$
 - closed set (optional): empty
2. While open set is not empty:
 - a. Pop (f, u) with smallest f from open set
 - b. If $u == t$: path found! Reconstruct and return.
 - c. Mark u as processed (add to closed set)
 - d. For each neighbor v of u with weight w :
 - $\text{tentative_g} = g[u] + w$
 - If $\text{tentative_g} < g[v]$:
 - $g[v] = \text{tentative_g}$
 - $f[v] = g[v] + h(v)$
 - $\text{parent}[v] = u$
 - Push $(f[v], v)$ into open set
 - (If v was in closed set, remove it – re-open)
3. If open set is empty and t not reached: no path exists.

7. Dry Run

Grid (4×4, 4-directional movement, each move costs 1):

```

S . . .
. # # .
. . . .
. . . T

```

S = start (0,0), T = target (3,3), # = wall (blocked)

Heuristic: Manhattan distance from (r,c) to (3,3): $h = |r-3| + |c-3|$

Step	Open Set (f, node)	Pop	g values updated	Notes
Init	(6, (0,0))	—	$g(0,0)=0$	$f(0,0)=0+h(0,0)=0+6=6$
1	(6, (0,0))	(0,0)	$g(0,1)=1, g(1,0)=1$	$f(0,1)=1+5=6, f(1,0)=1+5=6$
2	(6, (0,1)), (6, (1,0))	(0,1)	$g(0,2)=2$	$f(0,2)=2+4=6$
3	(6, (1,0)), (6, (0,2))	(1,0)	$g(2,0)=2$	$f(2,0)=2+4=6$
4	(6, (0,2)), (6, (2,0))	(0,2)	$g(0,3)=3$	$f(0,3)=3+3=6$
5	(6, (2,0)), (6, (0,3))	(2,0)	$g(3,0)=3, g(2,1)=3$	$f(3,0)=3+3=6, g(2,1)=3+2=5... wait$

Let me recalculate more carefully.

Manhattan distances to (3,3):

- (0,0): 6, (0,1): 5, (0,2): 4, (0,3): 3
- (1,0): 5, (1,1): wall, (1,2): wall, (1,3): 2
- (2,0): 4, (2,1): 3, (2,2): 2, (2,3): 1
- (3,0): 3, (3,1): 2, (3,2): 1, (3,3): 0

Step	Open Set (f, (r,c))	Pop	New g values
Init	(6, (0,0))	—	$g(0,0)=0$
1	(6, (0,0))	(0,0)	$g(0,1)=1, g(1,0)=1$
2	(6, (0,1)), (6, (1,0))	(0,1)	$g(0,2)=2$
3	(6, (0,2)), (6, (1,0))	(0,2)	$g(0,3)=3$
4	(6, (1,0)), (6, (0,3))	(1,0)	$g(2,0)=2$
5	(6, (0,3)), (6, (2,0))	(0,3)	$g(0,3)$ already = 3, no new neighbors (right is out of bounds)
6	(6, (2,0))	(2,0)	$g(3,0)=3, g(2,1)=3$
7	(5, (2,1)), (6, (3,0))	(2,1)	$f=3+2=5 \rightarrow g(2,2)=4, g(3,1)=4, g(2,0)=3$ (already smaller)
8	(5, (2,2)), (6, (3,0)), (5, (3,1))	(2,2)	$f=4+1=5 \rightarrow g(2,3)=5, g(3,2)=5, g(2,1)=3$ (already smaller)

Step	Open Set (f, (r,c))	Pop	New g values
9	(5, (3,1)), (5, (2,3)), (5, (3,2)), (6, (3,0))	(3,1)	$f=4+1=5 \rightarrow g(3,2)=\min(5, 4+1=5) = 5$, $g(3,0)=\min(3, 4+1=5) = 3$, $g(3,1)=4$
10	(5, (3,2)), (5, (2,3)), (6, (3,0))	(3,2)	$f=5+0=5 \rightarrow g(3,3)=6$, $g(3,1)=\min(4, 5+1=6)=4$
11	(6, (3,3)), (5, (2,3)), (6, (3,0))	(3,3)	Target found!

Found path! Distance = 6. Path: (0,0) $\rightarrow$ (1,0) $\rightarrow$ (2,0) $\rightarrow$ (2,1) $\rightarrow$ (2,2) $\rightarrow$ (3,2) $\rightarrow$ (3,3)

Comparison with Dijkstra: Dijkstra would explore roughly equally in all directions. A* focused on the direction toward the target, reducing the number of expanded nodes.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using pii = pair<int, int>;

const int INF = 1e9;

// Direction vectors for grid movement
const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

// Heuristic function: Manhattan distance
int heuristic(int r1, int c1, int r2, int c2) {
    return abs(r1 - r2) + abs(c1 - c2);
}

// A* Search on a grid
// grid[r][c] = 0 (walkable), 1 (wall)
// Returns shortest path distance and the path

struct Node {
    int f, g, r, c;
    bool operator>(const Node& other) const {
        return f > other.f;
    }
};

pair<int, vector<pii>> a_star_grid(const vector<vector<int>>& grid,
                                  pii start, pii end) {
    int rows = grid.size(), cols = grid[0].size();
    int sr = start.first, sc = start.second;
    int er = end.first, ec = end.second;

    // Distance arrays
    vector<vector<int>> g(rows, vector<int>(cols, INF));
    vector<vector<pii>> parent(rows, vector<pii>(cols, {-1, -1}));
```

```

vector<vector<bool>> closed(rows, vector<bool>(cols, false));

// Priority queue ordered by f = g + h
priority_queue<Node, vector<Node>, greater<Node>> pq;

g[sr][sc] = 0;
int h_start = heuristic(sr, sc, er, ec);
pq.push({h_start, 0, sr, sc});

while (!pq.empty()) {
    Node cur = pq.top();
    pq.pop();

    int r = cur.r, c = cur.c;

    // Skip if already processed with better g
    if (closed[r][c]) continue;
    closed[r][c] = true;

    // Goal reached
    if (r == er && c == ec) {
        // Reconstruct path
        vector<pii> path;
        int cr = r, cc = c;
        while (cr != -1) {
            path.push_back({cr, cc});
            auto [pr, pc] = parent[cr][cc];
            cr = pr;
            cc = pc;
        }
        reverse(path.begin(), path.end());
        return {g[r][c], path};
    }

    // Explore neighbors
    for (int d = 0; d < 4; d++) {
        int nr = r + dr[d], nc = c + dc[d];

        // Check bounds and walls
        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
        if (grid[nr][nc] == 1) continue; // Wall
        if (closed[nr][nc]) continue;

        int tentative_g = g[r][c] + 1; // Each move costs 1

        if (tentative_g < g[nr][nc]) {
            g[nr][nc] = tentative_g;
            int h = heuristic(nr, nc, er, ec);
            int f = tentative_g + h;
            parent[nr][nc] = {r, c};
            pq.push({f, tentative_g, nr, nc});
        }
    }
}

```

```

    // No path found
    return {INF, {}};
}

// Example usage
int main() {
    vector<vector<int>> grid = {
        {0, 0, 0, 0},
        {0, 1, 1, 0},
        {0, 0, 0, 0},
        {0, 0, 0, 0}
    };

    pii start = {0, 0}, end = {3, 3};

    auto [dist, path] = a_star_grid(grid, start, end);

    if (dist == INF) {
        cout << "No path found!\n";
    } else {
        cout << "Shortest distance: " << dist << "\n";
        cout << "Path: ";
        for (auto [r, c] : path) {
            cout << "(" << r << ", " << c << ") ";
        }
        cout << "\n";
    }

    return 0;
}

```

9. Python Implementation

```

import heapq
from typing import List, Tuple, Optional

# Direction vectors: up, down, left, right
dr = [-1, 1, 0, 0]
dc = [0, 0, -1, 1]

def heuristic(r1: int, c1: int, r2: int, c2: int) -> int:
    """Manhattan distance heuristic"""
    return abs(r1 - r2) + abs(c1 - c2)

def a_star_grid(grid: List[List[int]], start: Tuple[int, int], end: Tuple[int, int]):
    """
    A* search on a grid.

```

```

grid[r][c] = 0 (walkable), 1 (wall)
Returns (distance, path) where path is list of (r, c) tuples.
"""

rows, cols = len(grid), len(grid[0])
sr, sc = start
er, ec = end

# g-score: cost from start to node
g = [[float('inf')] * cols for _ in range(rows)]
parent = [[(-1, -1)] * cols for _ in range(rows)]
closed = [[False] * cols for _ in range(rows)]

g[sr][sc] = 0
h_start = heuristic(sr, sc, er, ec)

# Min-heap: (f, g, r, c)
pq = [(h_start, 0, sr, sc)]

while pq:
    f, cur_g, r, c = heapq.heappop(pq)

    if closed[r][c]:
        continue
    closed[r][c] = True

    # Goal reached
    if r == er and c == ec:
        # Reconstruct path
        path = []
        cr, cc = r, c
        while cr != -1:
            path.append((cr, cc))
            cr, cc = parent[cr][cc]
        path.reverse()
        return g[r][c], path

    # Explore neighbors
    for d in range(4):
        nr, nc = r + dr[d], c + dc[d]

        # Check bounds and walls
        if not (0 <= nr < rows and 0 <= nc < cols):
            continue
        if grid[nr][nc] == 1:
            continue
        if closed[nr][nc]:
            continue

        tentative_g = g[r][c] + 1 # Each move costs 1

        if tentative_g < g[nr][nc]:
            g[nr][nc] = tentative_g
            h = heuristic(nr, nc, er, ec)
            f = tentative_g + h

```

```

        parent[nr][nc] = (r, c)
        heapq.heappush(pq, (f, tentative_g, nr, nc))

# No path found
return (float('inf'), [])

# Example usage
if __name__ == "__main__":
    grid = [
        [0, 0, 0, 0],
        [0, 1, 1, 0],
        [0, 0, 0, 0],
        [0, 0, 0, 0]
    ]

    start = (0, 0)
    end = (3, 3)

    dist, path = a_star_grid(grid, start, end)

    if dist == float('inf'):
        print("No path found!")
    else:
        print(f"Shortest distance: {dist}")
        print(f"Path: {' → '.join(str(p) for p in path)}")

```

10. Code Explanation

- **g array:** Stores the actual shortest distance from start to each node. Initialized to INF, $g[start] = 0$.
- **parent array:** For path reconstruction. Stores the predecessor of each node.
- **closed set:** Nodes that have been fully processed (expanded). Once a node is closed, it won't be processed again (if heuristic is consistent).
- **Priority queue:** Min-heap ordered by $f = g + h$. We store (f, g, r, c) .
- **Heuristic function:** Manhattan distance to the goal. This is admissible (never overestimates) for grid movement.
- **Main loop:** Pop the node with smallest f . If it's the goal, reconstruct and return. Otherwise, explore neighbors.
- **Neighbor exploration:** For each valid neighbor, compute $tentative_g = g[u] + 1$. If this is better than the current $g[v]$, update and push to the open set.
- **Path reconstruction:** Follow parent pointers from goal back to start, then reverse.

11. Complexity Analysis

Aspect	Complexity
Time Complexity (worst case)	$O(E)$ — same as Dijkstra in worst case (when heuristic is poor)
Time Complexity (best case)	$O(b^d)$ where b = branching factor, d = depth to goal (when heuristic is perfect)

Aspect	Complexity
Time Complexity (with perfect heuristic)	O(d) — directly to the goal
Space Complexity	O(V) — stores open set, closed set, g-values
Practical Performance	Usually much faster than Dijkstra for large graphs with a good heuristic
• Worst case: When $h(n) = 0$ (degenerates to Dijkstra), A* explores all nodes. • Best case: When $h(n) = h^*(n)$ (perfect heuristic), A* goes straight to the goal. • Effective branching factor: The number of nodes expanded is typically much less than Dijkstra.	

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Grid pathfinding	2D grid, shortest path from start to end	Use Manhattan/Chebyshev heuristic	Shortest Path in Grid, Maze solving
8-puzzle / 15-puzzle	Sliding puzzle, heuristic for misplaced tiles	Number of misplaced tiles or Manhattan sum	Sliding Puzzle (LeetCode 773)
Robot motion planning	Configuration space, avoid obstacles	Euclidean distance heuristic	Robot path planning
GPS navigation	Road network, find route between two points	Euclidean distance or A* landmarks	Google Maps routing
Game AI	NPC pathfinding, RTS unit movement	Hierarchical A* (HPA*) or Jump Point Search	Game AI pathfinding
<i>A with weighted graph*</i>	Graph with arbitrary non-negative weights	Use edge weight in g(n) calculation	Route planning on weighted road network

13. Common Mistakes

- **Using a non-admissible heuristic** — If $h(n)$ overestimates, A* may not find the shortest path. Always ensure the heuristic is admissible.
- **Not checking if heuristic is consistent** — If h is not consistent, you may need to re-open closed nodes (which is expensive).
- **Forgetting to update g for better paths** — If a shorter path to a node is found, you must update g and push to the open set.
- **Not using a closed set** — Without a closed set, you may process the same node many times.
- **Using a closed set with an inconsistent heuristic** — You may miss a shorter path through a closed node if the heuristic is inconsistent.
- **Computing $h(n)$ too expensively** — If the heuristic is expensive to compute, it may negate the benefits.

- *Using A for all-nodes shortest path** — A* is designed for single-source single-target. Use Dijkstra for single-source all-targets.
- **Incorrect tie-breaking** — When f values are equal, the order of expansion matters for performance. Break ties by preferring larger g (closer to goal).

14. Edge Cases

- **Start = goal** → distance = 0, path = [start]
- **No path exists** → return INF / empty path
- **Start or goal is blocked** → return INF
- **Grid with all walls** → no path, handle gracefully
- **Single cell grid** → start = goal = that cell
- **Heuristic overestimates** → A* may return suboptimal path
- **Heuristic = 0** → A* degenerates to Dijkstra
- **Large open area with no obstacles** → A* still explores, but heuristic guides toward goal

15. Variations

Variation	What Changes	When Used	Importance
Weighted A*	Use $f = g + \epsilon \cdot h$ with $\epsilon > 1$	When you want a faster but possibly suboptimal path	Medium — trade accuracy for speed
<i>IDA (Iterative Deepening A)**</i>	Depth-first search with f-cost limit	When memory is tight (saves open set memory)	Medium — memory-efficient A*
<i>D (Dynamic A)**</i>	Handles dynamic changes in the graph	Robot navigation in unknown environments	High — robotics
Jump Point Search (JPS)	Optimized for uniform-cost grids	Grid pathfinding, much faster than A*	High — game AI
<i>HPA (Hierarchical A)**</i>	Hierarchical abstraction	Very large graphs (RTS games)	Medium — large-scale pathfinding
Bidirectional A*	Run A* from both start and goal	When both start and goal are known	High — reduces search space

16. Related Algorithms/Data Structures

Algorithm	Connection	When to Choose
Dijkstra	A* with $h=0$ is Dijkstra	Use Dijkstra for single-source all-targets or when no good heuristic exists
BFS	Unweighted graph, A* with $h=0$ behaves like BFS (with priority queue overhead)	Use BFS for unweighted graphs
Best-First Search	A* that ignores $g(n)$ (uses only $h(n)$)	Greedy, fast but not optimal

Algorithm	Connection	When to Choose
Greedy BFS	Uses only $h(n)$ to guide search	Very fast but not guaranteed to find shortest path
Bellman-Ford	Handles negative weights	Use when graph has negative weights

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Shortest Path in Binary Matrix	LeetCode 1091	Grid pathfinding, A* with Manhattan heuristic	Easy
Minimum Path Sum	LeetCode 64	Grid DP (simpler than A* for this constraint)	Easy

Medium

Problem	Platform	Pattern	Difficulty
Sliding Puzzle	LeetCode 773	15-puzzle, A* with misplaced tiles heuristic	Medium
Shortest Path in a Grid with Obstacles Elimination	LeetCode 1293	A* with state (k obstacles left)	Medium

Hard

Problem	Platform	Pattern	Difficulty
Minimum Cost to Reach Destination with K Flights	LeetCode 787	A* with heuristic for remaining budget	Hard
Robot in a Grid with Obstacles	GFG	A* pathfinding with obstacles	Hard

18. Interview Explanation

"A* is an informed search algorithm that finds the shortest path from a start node to a goal node. Unlike Dijkstra, which explores equally in all directions, A* uses a heuristic function $h(n)$ that estimates the distance from node n to the goal. The algorithm maintains a priority queue ordered by $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost from the start to n . At each step, it expands the node with the smallest $f(n)$. If the heuristic is admissible (never overestimates), A* is guaranteed to find the shortest path. With a good heuristic, A* can be much faster than Dijkstra. Common heuristics include Manhattan distance for grids and Euclidean distance for continuous spaces. The time complexity is $O(E)$ in the worst case, but in practice it's much faster."

19. Revision Notes

- **Key idea:** $f(n) = g(n) + h(n)$ — actual cost + heuristic estimate
- **Admissible heuristic:** Never overestimates (guarantees optimality)
- **Consistent heuristic:** $h(n) \leq c(n, m) + h(m)$ (guarantees no re-opening)
- **Common heuristics:** Manhattan (grid, 4-dir), Euclidean (any dir), Chebyshev (grid, 8-dir)
- **$h(n) = 0$:** Degenerates to Dijkstra
- $h(n) = h^*(n)$: Perfect heuristic, goes straight to goal
- **Time:** $O(E)$ worst case, much better in practice
- **Space:** $O(V)$ — open set + closed set
- **Use for:** Single pair shortest path with good heuristic
- **Not for:** Negative weights, all-pairs, no good heuristic

20. Final Cheat Sheet

A* SEARCH ALGORITHM

WHEN: Single pair shortest path, large graph, good heuristic available

COMPLEXITY: $O(E)$ worst case, much better in practice

DATA STRUCTURE: Min-heap priority queue (ordered by $f = g + h$)

TEMPLATE:

```

g[start] = 0
f[start] = h(start)
pq.push({f[start], start})
while (!pq.empty()):
    u = pq.pop()
    if (u == goal): return reconstruct_path()
    for v in neighbors(u):
        tentative_g = g[u] + w(u,v)
        if (tentative_g < g[v]):
            g[v] = tentative_g
            f[v] = g[v] + h(v)
            parent[v] = u
            pq.push({f[v], v})

```

KEY: $f(n) = g(n) + h(n)$

HEURISTIC: Must be admissible (never overestimate)

$h(n) = 0 \rightarrow$ Dijkstra

$h(n) = h^*(n) \rightarrow$ perfect, direct to goal

FAILS WITH: Negative weights, non-admissible heuristic

Comparison Table

Algorithm	Time	Space	Handles Negative Weights	Detects Negative Cycles	Use Case
BFS (Unweighted)	$O(V+E)$	$O(V)$	No (unweighted only)	No	Unweighted SSSP
Dijkstra	$O((V+E)\log V)$	$O(V)$	No	No	Non-negative weighted SSSP
Bellman-Ford	$O(V\cdot E)$	$O(V)$	Yes	Yes	Negative weights, cycle detection
Floyd-Warshall	$O(V^3)$	$O(V^2)$	Yes	Yes (detects)	Dense all-pairs
0-1 BFS	$O(V+E)$	$O(V)$	No (0/1 only)	No	0/1 weighted SSSP
DAG Shortest Path	$O(V+E)$	$O(V)$	Yes	N/A (DAG)	DAG SSSP
Johnson's	$O(V\cdot E + V^2 \log V)$	$O(V^2)$	Yes	Yes (detects)	Sparse all-pairs with negatives
A*	$O(E)$ worst	$O(V)$	No	No	Single pair with heuristic

Quick Decision Guide

Need shortest path?

- Single source → single target?
 - Have a good heuristic? → A*
 - No heuristic? → Dijkstra or BFS
- Single source → all targets?
 - Unweighted? → BFS
 - Weights 0 or 1? → 0-1 BFS
 - Non-negative weights? → Dijkstra
 - Any weights (including negative)? → Bellman-Ford
 - Graph is a DAG? → DAG Shortest Path ($O(V+E)$)
- All pairs?
 - Dense graph ($E \approx V^2$)? → Floyd-Warshall
 - Sparse graph with negatives? → Johnson's
 - Sparse graph without negatives? → Dijkstra × V
 - Unweighted? → BFS × V